B | HTTP 与 API

一个程序已经能整理工单，但如果工单来自网页，或者处理结果要交给另一个程序，单靠当前文件里的函数调用就不够了。对方可能运行在另一个进程、另一台电脑上，也不一定使用 Python。它需要知道：把什么送到哪里，会收到什么，失败时怎样分辨。

这一册学习的就是**两个程序之间怎样把事情交代清楚，并确认这次交接实际发生了什么**。先看一条完整请求和回应，再进入接收方的 Python 函数；随后从调用方检查结果，最后把两个服务连起来寻找失败位置。贯穿的例子是一个小型支持工单接口：创建工单、读取记录，以及请另一个服务生成一段摘要。

HTTP 是这里交换消息的约定；API 是一个程序对外提供能力的使用约定。两者的区别，会从第一条工单请求中展开。读完后，应当能看着一条调用说出：谁主动发起，谁处理，输入在哪，什么结果才可以继续用；有问题时先查哪一段，而不是只记住几个工具名字。

第一次只看这里

直接进入 B01。第一次只追一条交换：Python 值怎样变成 request，另一个程序怎样用 response 回答。

本册的详细关系图、章节路线和查用说明放在全书后部。遇到‘这一步和前后哪里相连’时再回看，不需要在第一轮先读完。

目录

第一次只看这里 1

B01 | 两个程序怎样交换一条工单？ 3

B02 | 请求怎样进入 Python 函数？ 13

B03 | 怎样只把可用回应交给后续代码 26

B04 | 两次 HTTP 交换怎样找到失败位置 40

B05 | 创建之后，怎样读回同一条记录？ 56

知识关系总览 | 把消息、函数和证据放回同一张图 66

需要继续查阅时 69

本册总览（需要时再回看） 71

本册知识结构总览 72

从哪里读到哪里 73

B01 | 两个程序怎样交换一条工单？

本章只追一个值：Python 里的标题和优先级，怎样离开当前进程变成请求，服务处理后又以可检查的结果回来。

一个 Python 程序已经能拿着标题和优先级处理工单。如果负责保存工单的是另一个正在运行的程序，当前程序该怎样把这些值交过去，又怎样知道对方处理的结果？

先把要观察的对象摆出来：浏览器里的 `/docs` 是服务公开出来的操作入口，Network 中的一条 `POST /tickets` 是实际发出的请求，右侧的响应 body 是服务交回的结果。三者属于同一次交换的不同观察位置；看到 `/docs` 页面本身，只能说明服务提供了文档入口，还不能说明工单已经创建。

先选本轮证据路线：

- **A | 先理解交换关系**：只看下面的纸面请求与响应，追踪“值 → request → response”。
- **B | 现在观察一次真实交换**：在自己的本地练习副本里打开服务和 `/docs`，只写入一条合成工单。

请先选 A 或 B；如果你还不知道副本和运行条件是否准备好，先选 A，不把“已经看过示意输出”当成真实运行证据。两条路线最后都会回到同一条值流，差别只是本轮有没有取得真实 Network / 服务日志。

先看见一次交换

如果你选择了 B，并且已经复制了 B 册的 `ticket-api` 实验副本，先不要输入命令。用资源管理器或编辑器只读确认副本中能看到 `app.py` 和 `.venv`：前者是服务代码，后者是运行它的项目环境。确认这两个对象都在，再只做下面第一个动作；如果缺少其中一个，就停在这里，回到配套说明准备副本。

第一步只启动服务：

1 在第一个 PowerShell 窗口启动服务：

```powershell
& .\.venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000
```

先看 Uvicorn 是否明确报告启动完成和 `http://127.0.0.1:8000` 的监听地址。窗口没有回到命令提示符，只说明命令尚未结束，不能单独证明正在监听；若出现报错，停在这里。启动日志正常后，再打开 `http://127.0.0.1:8000/docs`，用页面确实返回来核对请求能到达服务；这仍不等于工单已经创建。

提交前先按 F12 打开浏览器开发者工具，选“网络 / Network”，保持录制开启；它从此刻开始记录浏览器的请求。再回到 `/docs` 页面展开 `POST /tickets`，用 `Try it out` 编辑请求内容，填写标题 `Printer offline`、优先级 `2`，点击一次 `Execute`。

看到真实响应后，在 Network 列表中选中刚产生的 `/tickets` 请求，确认它的方法是 `POST`。此时只观察四件事：请求发往哪里、动作是什么、提交了什么、回应带回什么。若提交后才打开 Network，列表可能没有刚才的记录；不要为了抓图盲目再提交一次，因为那会再创建一条记录。可以保留已经返回的编号，按本章后面的 `GET /tickets/{id}` 读取它，观察这一次查询。

若 `.venv` 或服务还没准备好，先停在阅读模式；下面的纸面摘要可以帮助理解，但不算已经完成真实交换。完整的副本准备和安全的端口检查放在本册配套说明中，首次不需要先读完全部命令。

本章从“创建一条工单，再把它读回来”开始。先看两个程序真正交换的消息，再解释地址、动作、内容和结果分别放在哪里。以后写服务端或调用代码，都是在实现这里的交接。

一条创建请求，为什么要有一条回应？

假设电脑上已经运行着一个工单服务，地址是 `http://127.0.0.1:8000`。另一个程序希望创建标题为 `Printer offline`、优先级为 `2` 的工单。主动发起这次交接的程序叫客户端（client）；接收请求、处理并回答的程序叫服务端（server）。

它们不必运行在两台电脑上。同一台电脑的两个进程也可以各自承担这些角色；浏览器、命令行工具或 Python 程序都可以成为客户端。角色由这次交接决定。

随后再看下面这份便于阅读的 HTTP 请求（request）。它省略了长度等由工具处理的字段，重点保留当前需要解释的部分；纸面报文用于命名结构，不替代前面那次真实观察。

```HTTP
POST /tickets HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{"title":"Printer offline","priority":2}
```

服务创建成功后，用 HTTP 响应（response）回答：

```HTTP
HTTP/1.1 201 Created
Content-Type: application/json

{"id":"T-42","title":"Printer offline","priority":2,"status":"open","version":
  1}
```

先沿这一次变化看：客户端交去标题与优先级；服务端创建记录，分配编号 `T-42`，设定状态 `open`、版本 `1`；响应把这些公开信息带回。客户端此后可以用 `T-42` 指明要读取哪条工单。

请求和响应各有自己的组成：

消息	开头交代什么	headers（消息头）	body（消息体）
request	`POST` 表达动作，`/tickets` 指明目标	主机、内容类型等附加信息	希望提交的标题和优先级
response	`201 Created` 表达这次处理的结果	返回内容的类型等附加信息	新工单的公开资料

空行把消息头与消息体分开。`Content-Type: application/json` 声明这份 body 使用 JSON 表示内容。HTTP 工具会帮助生成和显示这些消息，通常不用手工拼整份报文。

这里有两个容易混在一起的“状态”：`201` 是响应的 HTTP 状态码，说明这次创建请求成功；body 中的 `"status":"open"` 是工单当前的业务状态。以后把工单改成 `closed`，也可能得到 HTTP 成功状态码。

一条请求的处理结果，与被处理对象自己的状态，是两件事。 服务端需要把它们分别表达，客户端才能知道请求是否完成、资源现在是什么样。

HTTP、API 和 JSON 各自负责什么？

这份服务允许外部程序通过 `POST /tickets` 创建工单，并约定需要哪些字段、会返回什么。这套让别的程序使用服务能力的约定，就是应用程序接口（application programming interface，API）。接口不只是一个网址；动作、输入、结果和失败方式也属于约定。

HTTP 是这次 API 使用的消息交换协议，约定请求与响应怎样表达。JSON 是 body 中的数据表示格式，约定字符串、数字、数组和对象怎样写。它们在这次交接中配合工作：

工单 API 约定：如何创建工单，需要哪些字段，成功后交回什么

↓ 使用 HTTP 表达这次调用

POST /tickets + headers + body
JSON 表示提交数据

→ 服务

→ status + headers + body
JSON 表示返回数据

HTTP 也可以传送网页、图片和普通文本，不要求每份 body 都是 JSON。API 也可以采取其他交接形式，不是所有 API 都使用 HTTP。本书讨论的是使用 HTTP 的 API，并选择 JSON 传送这些结构化字段。

这就解释了为什么不能只给另一个程序一段 JSON：JSON 能表示标题和优先级，却没有单独回答“发给谁、希望做什么、对方怎样说明结果”。这些关系由地址、HTTP 消息和具体 API 约定共同补上。

> **第一次学到这里先停。** 你现在只需要能追踪一个值：Python 里的标题和优先级，经过请求交给服务，服务把编号、业务状态和处理结果放回响应。先确认这条交接关系；序列化细节、完整地址变化、状态码边界和 Network 查法，都是在这条主线上继续加证据，不是现在要一次记完的命令表。

查用：Python 字典看起来像 JSON，为什么还要转换？

这段解释的是库在请求边界替你完成的表示转换。第一次阅读只要保留“当前进程里的 dict → 请求 body → 对方重新得到自己的值”；不必先记住 `bytes`、编码和解码的完整写法。

客户端最初可能拿着这样的 Python 值：

PYTHON

```
ticket_input = {
    "title": "Printer offline",
    "priority": 2,
}
```

这是当前进程内存中的 `dict`。同一个程序里的函数可以直接接收这个对象，另一个进程却不能仅凭变量名 `ticket_input` 取得它。双方需要能传送、能重新解释的表示。

把值转换成约定表示的过程叫序列化（serialization）；接收方按格式规则读出值叫解析（parsing）。下面的本地小例把这两个动作单独展示出来，它本身还没有发送请求：

```python
import json

ticket_input = {"title": "Printer offline", "priority": 2}
json_text = json.dumps(ticket_input)
body_bytes = json_text.encode("utf-8")

received_text = body_bytes.decode("utf-8")
received_value = json.loads(received_text)

print(type(json_text))
print(type(body_bytes))
print(received_value["priority"])
```

输出是 `<class 'str'>`、`<class 'bytes'>`、`2`；前两行分别说明当前值是文字和字节。`json.dumps` 产生 JSON 文本，`encode` 把文字按 UTF-8 编成字节；反方向先解码，再用 `json.loads` 解析出新的 Python 值。在真正 HTTP 调用中，客户端库与服务端框架会承担其中很多步骤。

> 客户端 dict → JSON 表示的 body 字节 → 服务端解析出的本地值
> 服务端结果 → JSON 表示的 body 字节 → 客户端解析出的新值

传过去的是表示，接收方重新得到的是自己的值。 两端信息可以对应，不意味着同一个字典对象在两个进程之间移动。客户端修改自己收到的字典，也不会自动改掉服务端保存的工单。

有些 JSON 写法与 Python 明显不同。为了只看格式，再用一份不发给工单接口的小数据：

```python
value = {"confirmed": True, "owner": None}
text = json.dumps(value)
print(text)
```

结果是：

```json
{"confirmed": true, "owner": null}
```

Python 的 `True` 和 `None` 在 JSON 中写成 `true` 和 `null`；解析回来又成为 Python 的 `True` 和 `None`。JSON 对象的键和字符串使用双引号，不能把 Python 字典的任意显示文字当成合法 JSON。

这里选择 `json.dumps`，是为了让格式库处理引号、转义和这些值的映射。用字符串拼接把标题塞进大括号，标题一旦含双引号或换行，就容易破坏格式。后面使用 HTTPX 时，`json=...` 可以直接接收 Python 值并完成 JSON 编码。

解析成功也不代表一定得到字典。JSON 的顶层可以是数组、字符串、数字、布尔值或 `null`。当前工单接口约定返回 JSON object，客户端仍需按这个约定检查解析结果。

完整地址怎样选中刚才那条工单？

创建后，客户端使用返回的编号读取记录：

> http://127.0.0.1:8000/tickets/T-42?include_history=true

这份 URL 把几个不同问题组合起来：

部分	当前写法	在这里说明什么
scheme	`http`	使用 HTTP；`https` 则是在加密保护的连接上使用 HTTP
host	`127.0.0.1`	目标是当前电脑自己
port	`8000`	这台电脑上的服务监听入口
path	`/tickets/T-42`	指向工单 `T-42`
query	`include_history=true`	这一次读取还希望包含历史记录

`127.0.0.1` 属于回环地址（loopback）：这里的通信在本机往返。换到另一台电脑输入同一地址，指向的是另一台电脑自身，不会继续指向原先那台电脑。

端口帮助把请求交到主机上相应的监听服务。到达服务后，`/tickets/T-42` 再由服务解释。这里的 path 不是磁盘路径；请求这个 URL 不要求电脑上存在一个叫 `tickets` 的文件夹。

读取对应的请求是：

```http
GET /tickets/T-42?include_history=true HTTP/1.1
Host: 127.0.0.1:8000
```

这次没有提交创建数据，因而不需要那份 JSON body。服务返回 `200 OK`，内容是：

```json
{
  "id": "T-42",
  "title": "Printer offline",
  "priority": 2,
  "status": "open",
  "version": 1,
  "history": ["created"]
}
```

若省略 `?include_history=true`，仍读取 `T-42`，只是本例响应不包含 `history`。这就把 path 和 query 的区别落到结果上：前者选中工单，后者调整这一次怎样读取它。

地址中的 `true` 此刻是 URL 里的文字，不是 Python 布尔值。服务端读取并转换后，才会让函数得到 `True`。下一章会打开这一步。

为什么创建用 POST，读取用 GET?

只知道资源地址还不够，服务需要知道客户端希望对它做什么。HTTP method（方法）提供通用动作语义；具体 API 再规定每个动作在哪些路径可用。

`GET /tickets/T-42` 请求读取当前表示，`POST /tickets` 则在本例中提交一条新工单。这两项工作都涉及工单，但预期效果不同。

现在原样再发一次创建请求。当前服务会创建另一条工单，编号继续增加；相同标题和优先级不会让服务自动认定它与上次属于同一次操作。原样再读一次 `T-42`，则不应因为读取动作而把工单关闭或新建一条记录。

这个区别也帮助理解两个常见词。安全方法（safe）指请求的动作本身不应改变资源状态，GET 属于这一类；它不代表信息保密，服务仍可能记录访问日志。幂等（idempotent）指重复相同请求意图，对目标资源的预期效果与执行一次相同；它不要求每次回应的字节或状态码完全相同。

除 GET 和 POST 外，阅读 API 文档还会遇到：

method	通常表达的意图	与当前例子的关系
PUT	用提交的表示完整替换目标资源	本地配套没有实现，不能直接套用地址调用
PATCH	按接口约定局部修改资源	配套用它更新工单状态
DELETE	删除指定资源	本地配套没有实现

GET、PUT、DELETE 在 HTTP 语义上是幂等的；POST、PATCH 不提供普遍的幂等保证。具体副作用还需读接口约定。现在先能区分读取与创建的效果；遇到“请求结果不确定，要不要再发一次”时，后续可靠性章节会继续处理这个问题。

把所有动作都写成 POST 也能被某些 API 采用，但那样动作的线索主要藏在自定内容中。当前采用常见 method 与资源路径的组合，使调用者与观察工具在消息开头就能看出读、建、改的意图。这不是看到一个英文动词就能猜出所有行为，接口文档仍然决定细节。

状态码成功，正文就一定能用了么？

创建用 `201 Created`，读取用 `200 OK`，都是成功类状态。其他情况需要结合具体数字和内容读：

范围	大致表达什么	一个具体例子
2xx	请求成功处理	201 创建成功
3xx	重定向等后续处理信息	302 常指向另一个位置，可查看 `Location` header
4xx	当前请求、身份或目标不满足要求	404 未找到；本地输入校验还会用到 422
5xx	服务未正常完成处理	503 服务当前不可用

这些分类帮助确定下一步查看什么，不替代具体调查。`GET /ticket/T-42` 把路径写错了，也可能得到 404；`GET /tickets/T-404` 路径形状正确、工单却不存在，也可能得到 404。

再看两个都返回 `200` 的响应：

```HTTP
HTTP/1.1 200 OK
Content-Type: application/json

{"status":"open"}
```

```HTTP
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8

status=open
```

第一份声明 JSON，第二份声明普通文本，二者的解析方式不同。`Content-Type` 是发送方关于内容的声明，也可能声明错；即使写着 JSON，实际 body 仍可能损坏。反过来，某些标为文本的 body 恰好能按 JSON 解析，也不能因此忽略当前 API 对格式的约定。

status、内容类型和实际内容要接着检查，任何一项都不能独自证明结果可用。 后面客户端会把这几次检查写成顺序明确的代码。当前只需能指出：200 已经证明收到一份成功类 HTTP 响应，尚未证明其中有程序需要的工单字段。

HTTP 状态码也不同于进程退出码。一个长期运行的服务可以连续回答多次请求，各自返回 201、404 或 422，而服务进程继续运行。一个命令行客户端最后退出时，才通过自己的退出码说明该次程序运行的结果。

Header 里除了内容类型，还可能有什么？

body 承载本次主要数据，header 提供解释和处理消息的附加信息。请求中的 `Accept: application/json` 表达客户端希望接收 JSON；响应中的 `Content-Type` 声明服务实际送回的类型。表达希望不会强制服务满足它。

身份相关信息也常出现在请求头：

```HTTP
Authorization: Bearer <token>
Cookie: session=<value>
```

`Authorization` 携带按某种认证方案使用的凭据。Cookie 则常由浏览器保存，并按域、路径等规则随请求带回；服务可以用其中的会话标识认出此前的访问。这里的尖括号只是占位，并非可使用的凭据；当前本地创建和读取不需要它们。

这些 header 不代替工单标题，也不代替响应状态码。观察请求时可以确认它们位于哪里，但分享截图、日志或复制的请求前，应去掉真实 token、会话值和敏感正文。

页面写着“创建成功”，怎样看到实际交换？

页面上的一句文字是界面最终选择显示的内容。想知道它发给谁、带了什么、收到什么，需要查看浏览器开发者工具中的 Network（网络）记录。

前面的报文已经给足纸面理解所需的输入和结果。实际观察时，先到 B05 的“在电脑上，先把项目放到哪里？”复制项目；再按“怎样准备同一个 Python 环境？”建立 `.venv`、安装依赖，做到 `pip check` 通过。随后只做“为什么要打开两个窗口？”中的端口检查与 8000 服务启动。看到运行地址后，保持服务窗口开着就回到这里；测试、第二个 PowerShell 窗口、客户端脚本和 8001 供应商服务都留到 B05 完整操作时再做。

现在在浏览器打开 `http://127.0.0.1:8000/docs`。这个页面是根据接口说明生成的交互文档，可以代替自己编写网页来发送请求。

先按 `F12` 打开开发者工具，选择 Network，保持记录开启。展开文档中的 `POST /tickets`，进入可填写参数的操作区，提交：

```JSON
{"title":"Printer offline","priority":2}
```

然后在 Network 中选中刚产生的 `/tickets` 请求。加载 `/docs` 页面本身、加载 `/openapi.json` 说明文件也会有请求；当前要看的，是 method 为 POST 的创建操作。

界面分区名称可能略有不同，按其承担的工作寻找：

观察位置	本次具体核对什么
Headers 中的概况	Request URL 是否指向本机 `/tickets`，Request Method 是否为 POST，Status Code 是否为 201
Request Headers	提交内容是否声明为 JSON
Payload 或请求内容区	`title` 和 `priority` 是否与刚填的一致
Response Headers	服务声明返回什么内容类型
Response	实际返回哪个编号，以及其他公开字段

取这次实际返回的 `id`，再用文档中的 GET 操作读取它。新启动的服务第一次创建是 `T-42`；之前创建过记录时，编号会继续增加，应使用自己的返回值。

浏览器地址栏适合发起一次读取，不能仅靠在地址栏粘贴 URL 完成带 JSON body 的 POST。交互文档方便手工构造请求，Network 帮助观察真实交换；HTTPX 则让 Python 在程序内部完成相同交接。工具不同，消息里的目标、动作、输入与结果仍可一一对应。

到这里，创建请求的标题和优先级已经变成带编号的公开结果，再由编号引出读取请求。后面会反复用到读完整地址、区分请求与响应、查看状态和内容这些动作；它们需要逐渐用顺。完整状态码表和工具按钮可以查，核心是能解释自己现在查的是什么。

下一章进入服务端：请求到达后，谁按 `POST /tickets` 找到 Python 函数，谁把 JSON 字段变成函数能使用的值，又是谁决定向客户端返回什么。

B02 | 请求怎样进入 Python 函数?

客户端向 `POST /tickets` 提交标题和优先级，服务端返回了新工单。但请求没有写出 Python 函数名，也没有直接传入一个 Python 字典。服务怎样知道该运行哪段代码?

这一章先让一条创建请求进入一个小程序，再沿着读回工单、输入错误和状态修改，看看服务如何取值、决定是否继续以及形成公开结果。HTTP 消息与 Python 函数之间的连接会逐步展开。

先让一条请求找到一段代码

这里使用 FastAPI 编写服务。框架（framework）提供接收与处理请求的一套组织方式，我们在指定位置写业务函数。FastAPI 会按请求找到函数，检查输入，再把函数结果组织成响应。

下面是一份独立最小程序，可保存为 `mini_api.py`。它只演示创建工单，不是要求替换完整配套的 `app.py`:

PYTHON

```
from fastapi import FastAPI
from pydantic import BaseModel, Field


class CreateTicket(BaseModel):
    title: str = Field(min_length=3, max_length=120)
    priority: int = Field(ge=1, le=3)


app = FastAPI()
app.state.tickets = {}
app.state.next_ticket_number = 42


@app.post("/tickets", status_code=201)
def create_ticket(payload: CreateTicket):
    ticket_id = f"T-{app.state.next_ticket_number}"
    app.state.next_ticket_number += 1
    record = {
        "id": ticket_id,
        "title": payload.title,
        "priority": payload.priority,
        "status": "open",
        "version": 1,
    }
    app.state.tickets[ticket_id] = record
    return record
```

先把代码外面的动作与里面的动作对应起来：

```HTTP
POST /tickets HTTP/1.1
Content-Type: application/json

{"title":"Printer offline","priority":2}
```

这条请求会匹配 `@app.post("/tickets", status_code=201)` 登记的入口。输入通过检查后，FastAPI 调用下面的 `create_ticket` 函数；第一次调用创建 `T-42`，响应状态为 201，body 为：

```JSON
{"id":"T-42","title":"Printer offline","priority":2,"status":"open","version":
  1}
```

现在逐项恢复第一次出现的写法。两个 `from ... import ...` 从已安装的包中取得要用的工具。`FastAPI()` 建立应用对象，赋给变量 `app`；它保存我们登记的接口等信息。`app.state` 提供一个存放应用状态的位置，这里放入工单字典和下一个编号。

这些值属于当前运行进程的内存。再次收到请求时，同一个应用可以继续使用它们；停止服务、重新加载这份程序后，又从空字典和编号 42 开始。`app.state` 没有自动把记录保存到磁盘。

`class CreateTicket(BaseModel):` 定义一种输入模型。`BaseModel` 来自 Pydantic，它让模型能按声明检查字段。当前可以先把这块读成“创建工单输入的具体约定”，不必先推演类的内部实现。

`title: str` 指出标题应为字符串，`Field(min_length=3, max_length=120)` 再限制长度。`priority: int` 指出需要整数，`ge=1` 表示至少为 1，`le=3` 表示至多为 3；没有给字段默认值，因此两项都必须提供。

普通 Python 类型注解不会自动检查任意函数调用。这里能发生输入检查，是因为 FastAPI 读取了这些声明，并使用 Pydantic 处理请求数据。工具使用了声明，声明才在这次运行里产生作用。

`@app.post(...)` 是装饰器写法，在这里负责把 method POST、path `/tickets` 与紧随其后的函数登记为一条 route（路由）。程序启动、读到这段定义时是在登记入口；不是每读到一次 `def` 就创建一条工单。

请求匹配以后，`payload: CreateTicket` 告诉框架用这个模型接收 body。框架解析 JSON、检查字段，然后把一个 `CreateTicket` 对象交给参数 `payload`。因此函数用属性 `payload.title` 和 `payload.priority` 取值，而不是从原始 JSON 文本里切字符串。

> body 的 title 字段 "Printer offline" → 检查 → payload.title
> body 的 priority 字段 2 → 检查 → payload.priority

函数体中的 f-string 用当前编号 42 形成 `T-42`，下一行把计数增加为 43。`record` 保存这条工单，`app.state.tickets[ticket_id] = record` 再按编号登记它。最后 `return record` 把 Python 结果交给框架，框架把字典序列化为 JSON，按 route 的设定返回 201。

endpoint 是框架按请求调用的函数；它仍然通过参数取得值，通过 return 交回结果。 endpoint 常译为接口端点，这里指处理当前请求的入口函数。仅把函数命名为 `create_ticket` 不会让网络调用者找到它，真正的入口来自 method 和 path 的登记。

如果函数只有 `print(record)`，打印的文字会出现在服务所在的输出位置，不会自动成为客户端需要的 JSON 结果。这与本地函数里 `print` 和 `return` 的区别相连：结果要交给谁，决定该用哪一种交接。

FastAPI 和启动服务的工具是什么关系？

把上面的文件保存好，还不等于端口已经有人接收请求。实际运行时，Uvicorn 加载这个应用，监听指定地址，把接收的请求交给 FastAPI。FastAPI 再按 route 调用业务函数，并把响应交回。

> 客户端 → 本机监听端口 → Uvicorn → FastAPI → create_ticket
> 客户端 ← HTTP response ← Uvicorn ← FastAPI ← return record

已有环境和独立示例文件时，启动入口可以写成：

```powershell
& .\.venv\Scripts\python.exe -m uvicorn mini_api:app --host 127.0.0.1 --port
  8000
```

`mini_api:app` 前半部分是模块名，后半部分是该模块里的应用对象名。它不是磁盘上一个叫 `mini_api:app` 的文件。命令假定已经按 B05 准备 `.venv`，并把 `mini_api.py` 存在该实验目录；这个 Python 必须能导入 FastAPI、Pydantic 和 Uvicorn。两个不同服务不能同时占用同一个监听地址，运行小例前也要按 B05 的方法确认端口空闲。

工具各自承担不同工作：Uvicorn 让应用参与实际网络交换；FastAPI 管理 HTTP 入口、输入和结果；我们写的函数决定工单怎样变化。直接调用 `create_ticket(...)` 可以研究函数内部行为，却没有检验 HTTP 路由、JSON 解析和响应形成。

为什么先检查输入，再运行创建函数？

假如客户端把优先级写成了文字：

```JSON
{"title":"Printer offline","priority":"urgent"}
```

它仍是合法 JSON，但 `"urgent"` 无法成为创建工单所需的整数。请求的先后关系是：

> POST /tickets 匹配成功
> → body 能按 JSON 解析
> → CreateTicket 检查 priority
> → "urgent" 不能成为所需整数
> → 返回 422
> → create_ticket 没有开始执行

FastAPI 默认用 422 表达这类请求校验失败，并在错误内容中指出字段位置和原因。完整配套还会记录必要的关联信息。无论错误内容怎样展示，本例最重要的效果是：编号没有增加，工单字典没有新增记录。

先解析，再校验，这两项工作不能互相代替。`{"priority":}` 连 JSON 都不完整；`{"priority":"urgent"}` 能解析，却不满足字段要求。标题太短、优先级超出范围或缺少必需字段，也会被输入模型拒绝。

这里的整数检查不是“任何 JSON 字符串都不允许”。Pydantic 默认可以转换某些合法表示，例如数字文字；本例特意使用无法转成整数的 `"urgent"`。若接口确实需要严格拒绝转换，需要另行明确声明严格规则。

也可以让函数接收一个普通字典，再逐个判断有没有字段、能否转换和是否超出范围。这个办法在很小的内部操作里可能直接；当前输入要同时被外部调用者、接口文档和测试理解，用模型把稳定约定集中起来更清楚。它减少了每个 endpoint 重写同一套基础检查的机会。

模型并不知道所有业务事实。`T-404` 可以是形式合法的编号，却在工单字典中不存在；这需要执行查找以后才能知道。

接口文档怎么知道要填写哪些字段？

前一章打开的 `/docs` 页面不是凭空猜测输入。FastAPI 会把登记的 route、参数、输入模型和输出声明整理成一份 OpenAPI 说明，默认可在 `/openapi.json` 取得；交互文档读取这份说明，才知道有哪些操作、需要填什么、应展示什么成功结果。

这里会遇到 schema（结构约定）这个词。对于创建工单，它描述 `title` 是必需字符串、长度在 3–120，`priority` 是必需整数、范围在 1–3。它描述的是一类允许的数据，不是某次提交的 `{"title":"Printer offline","priority":2}`。OpenAPI 在这些数据结构外，还把 method、path、参数位置与回应说明连接起来，让别的工具也能读取接口约定。

当前用输入模型集中声明字段，除了运行时检查，还能减少文档和代码各写一份、后来只改一边的机会。没有必要手写整份 OpenAPI 文件或记住每个 schema 字段；应能从文档找出这次调用的目标、输入要求与成功结果，再用实际请求核对。说明文件反映声明，并不保证业务代码、部署状态或每条实际回应都正确。

URL 里的编号和选项怎样进入函数？

创建后，客户端读取：

```
GET /tickets/T-42?include_history=true
```

完整配套把 GET 与 `/tickets/{ticket_id}` 登记为读取 route，使用下面的函数签名：

PYTHON
```python
def get_ticket(
    ticket_id: str,
    response: Response,
    include_history: bool = False,
):
```

这些参数虽然出现在同一个函数里，来源却不同。`ticket_id` 的名字出现在 route 的 `{ticket_id}` 中，因此从 path 接收 `T-42`。`include_history` 没有出现在 path，也不是一个 body model，在这里作为 query 参数接收文字 `true`，转换为 Python 的 `True`；省略 query 时使用默认值 `False`。

`response: Response` 又不同。`Response` 从 FastAPI 导入，框架提供这个响应对象，让函数可以增加返回的 header。客户端不需要在 URL 中再塞一个叫 `response` 的值。

下面是配套读取函数处理记录的部分，其中 `HTTPException` 同样来自 FastAPI：

```python
record = app.state.tickets.get(ticket_id)
if record is None:
    raise HTTPException(status_code=404, detail="ticket_not_found")

history = record.get("history", [])
response.headers["X-History-Count"] = (
    str(len(history)) if include_history else "not-requested"
)
public_record = dict(record)
public_record["history"] = list(history) if include_history else None
return public_record
```

配套创建时会在内部记录加上 `"history": ["created"]`。因此 `include_history=True` 时，这段代码把历史放进返回副本，并把响应头 `X-History-Count` 设为文字 `"1"`。未请求历史时，副本中该项先设为 `None`，读取 route 的输出设置会让它不出现在最终 JSON 中。

`a if condition else b` 是在两个值之间选择的写法：条件成立取左边的 `a`，否则取右边的 `b`。所以 `list(history) if include_history else None` 在请求历史时产生列表，没有请求时产生 `None`；选择出的值才赋给 `public_record["history"]`。它没有让整个函数提前结束。

`dict(record)` 在这里建立一份浅拷贝，`list(history)` 也为返回历史建立一个列表副本。这让当前读取选择作用在准备返回的对象上，不是为了省略历史就删除服务保存的历史。

这些输入位置帮助调用者区分三件事：path 指明这次处理谁，query 调整这次怎样读取，body 带来要创建或修改的结构化数据。它们不是在比较谁能放更多文字，而是在把不同职责表达清楚。当前没有必要把整份工单塞到 URL 中。

得到 404，说明函数没有运行吗？

不一定。请求 `GET /tickets/T-404` 时，可以匹配读取 route，`ticket_id` 也能接住字符串。函数开始执行，查找得到 `None`，才主动抛出 `HTTPException`。

匹配 /tickets/{ticket_id}
→ ticket_id = "T-404"
→ get_ticket 开始
→ 字典中没有这个编号
→ raise HTTPException(404, ...)
→ 框架形成 404 与 {"detail":"ticket_not_found"}

`raise` 让函数后续不再照常执行，FastAPI 接住这个约定的 HTTP 错误并形成响应。这是主动表达业务结果，不是服务进程因此崩溃。

若请求误写成 `GET /ticket/T-404`，没有匹配这条 route，也可能得到 404。但这次 `get_ticket` 根本没运行。相同状态码可以由不同位置产生，判断需要把路径、错误内容和相关运行记录放在一起。

在没有输出模型约束的简单 route 中，如果只 `return {"detail": "ticket_not_found"}`，却保留默认成功状态码，就可能让客户端把失败看成 HTTP 成功。完整配套的读取 route 还有输出模型，这个不含工单字段的字典会在输出检查时失败，后面会具体说明。选择 `HTTPException`，是为了明确把失败状态与可公开说明一起交给框架；body 中写了一句错误，并不会自动形成预期的 404。

输入都合法了，为什么还可能不能修改?

现在 `T-42` 的状态是 `open`，版本是 `1`。客户端希望改成 `closed`，同时提交自己根据哪个版本作出这次修改:

```http
PATCH /tickets/T-42/status HTTP/1.1
Content-Type: application/json

{"status":"closed","expected_version":1}
```

配套用下面的模型检查修改输入:

```python
from typing import Literal


class UpdateStatus(BaseModel):
    status: Literal["open", "pending", "closed"]
    expected_version: int = Field(ge=1)
```

`Literal[...]` 把状态限制为列出的文字，其他状态会在函数以前被拒绝。`expected_version` 必须能成为至少为 1 的整数；这一步只检查值的形式与范围，还没有与当前工单比较。

匹配 `PATCH /tickets/{ticket_id}/status` 的函数接收 path 中的 `ticket_id` 和 body model `payload`，再执行：

```python
record = app.state.tickets.get(ticket_id)
if record is None:
    raise HTTPException(status_code=404, detail="ticket_not_found")
if payload.expected_version != record["version"]:
    raise HTTPException(status_code=409, detail="version_conflict")

record["status"] = payload.status
record["version"] = int(record["version"]) + 1
record["history"].append(f"status:{payload.status}")
return record
```

提交版本 1 与当前版本 1 相等，函数才把状态改成 `closed`、版本增加为 2，并追加历史。客户端得到的公开结果包括新版本。

另从同样的初始状态 `open`、版本 1 看一次失败：如果提交的是 `expected_version=7`，这个正整数仍通过输入模型；但函数读到当前版本 1 后发现不相等，形成 409 `version_conflict`。写入行还没有执行，工单保持原样。

这个规则的动机是防止调用者在依据与当前状态不一致时直接改写。比如另一项修改已把版本变成 2，仍拿着版本 1 的调用者可以被明确告知需要先重新读取。版本必须公开给调用者，它才有东西可在下一次请求中提交。

输入检查判断这份值能否作为请求使用，业务判断还要读取当前事实，决定这次操作是否成立。 422、404、409 在这里分别暴露了不同检查的位置。

这段内存代码用于解释先比较、后修改的控制顺序。比较和写入没有作为一个不可分割操作受到并发保护，不能据此宣称两个同时到来的修改一定只会成功一个。后续把状态放进数据库时，还需要数据库提供相应的原子操作或事务保证。

函数 return 的内容，为什么不全部交给客户端？

完整配套内部记录除了公开工单字段，还有：

```python
"internal_note": "created-by-local-training-api",
"history": ["created"],
```

内部备注用于服务自己的工作，创建响应并不需要公开它。若直接依赖每个函数临时删除字段，后来增加一个内部字段时，可能在某个返回位置忘记处理。

配套把创建和修改的公开结果集中声明为：

```python
class TicketPublic(BaseModel):
    id: str
    title: str
    priority: int
    status: str
    version: int
```

创建 route 使用这个模型：

```python
@app.post("/tickets", response_model=TicketPublic, status_code=201)
```

函数仍可以 `return record`。FastAPI 在它返回后按 `TicketPublic` 检查与处理结果，创建响应只包含声明的五个字段。内部 `record` 的备注和历史没有因此被删除，改变的是对外交接的表示。

request body	→ **服务端** CreateTicket 检查 ↓ endpoint 操作内部记录 ↓ return 内部结果 ↓ TicketPublic 检查与过滤	→ 客户端 JSON

读取 route 需要按 query 选择是否给出历史，因此配套另有 `TicketDetail`，在相同字段之外声明 `history: list[str] | None = None`。`list[str]` 表示字符串列表，`| None` 允许没有历史值。route 上的 `response_model_exclude_none=True` 让值为 `None` 的字段从响应省略；因此不请求历史时，客户端看到的是没有 `history`，并非 `"history": null`。

输出 model 同时帮助接口文档描述成功结果。它适合稳定的公开形状；很小的函数也可以明确构造一份公开字典。但只声明字段并不替代权限判断，哪些人能读取哪些工单仍是另一项工作。

输入模型和输出模型虽然都用 Pydantic，却位于函数两边：前者约束调用者交来的值，后者约束服务承诺交出去的值。不能因为输入已经检查过，就认为返回结果自然正确。

服务自己返回错了字段，为什么不是 422？

假设代码修改后，在声明了 `TicketPublic` 的创建函数里只返回：

PYTHON
```
return {"title": payload.title, "priority": payload.priority}
```

输入可能完全合法，函数也可能已经执行。但公开结果缺少 `id`、`status` 和 `version`，无法满足服务自己声明的输出模型。FastAPI 会产生响应校验错误，默认正常服务运行下向调用者返回 500，而不是把这份不合约数据作为成功结果交出去。

422 指向本例的请求输入问题；这次应调查服务端代码怎样构造返回值。不能让客户端去修改一个本来合法的标题，替服务补上缺失的编号。

输出检查失败也不等于此前写入自动撤销。如果函数先把记录放进字典，再交回不合约结果，输出检查在写入以后才发生。客户端没有得到可用成功响应，服务端可能已经产生副作用。这是理解“收到失败是否可以再创建一次”之前必须保留的事实。

开发测试时，TestClient 默认会把服务端异常直接抛到测试进程，使人容易看到原因。若特意检查外部响应，可用 `raise_server_exceptions=False` 观察 500。两种观察方式对应同一个服务错误，并不是服务器在测试里把它判成了另一种业务结果。

怎样检查整个接口，而不只检查函数？

直接调用一个 Python 函数，能检查它接收值以后怎样工作；却绕过了 method/path 匹配、body 转换、输入校验和响应输出。这里需要从请求形状出发观察结果。

FastAPI 的 TestClient 可以在测试进程里把这种请求交给应用，不必先启动真正的监听端口。下面针对开头的独立 `mini_api.py` 建立一个测试文件 `test_mini_api.py`：

PYTHON
```
from fastapi.testclient import TestClient

from mini_api import app


def test_create_and_reject_invalid_input():
    app.state.tickets = {}
    app.state.next_ticket_number = 42

    with TestClient(app) as client:
        created = client.post(
            "/tickets",
            json={"title": "Printer offline", "priority": 2},
        )
        assert created.status_code == 201
```

PYTHON · 续

```
        assert created.json() == {
            "id": "T-42",
            "title": "Printer offline",
            "priority": 2,
            "status": "open",
            "version": 1,
        }

        rejected = client.post(
            "/tickets",
            json={"title": "Printer offline", "priority": "urgent"},
        )
        assert rejected.status_code == 422
        assert set(app.state.tickets) == {"T-42"}
        assert app.state.next_ticket_number == 43
```

这是独立测试进程中的应用，开头清空的是这个小例的测试状态。`client.post` 按请求位置提交 JSON；返回对象提供 `status_code` 和 `json()`，让断言把实际结果与预期比较。`with` 管理客户端使用区间，结束后退出这个上下文。

`set(app.state.tickets)` 取出这份字典的键，形成不关心顺序的集合；与 `{"T-42"}` 比较，就是确认当前只有这一条编号。这里不是再次创建一个工单字典。

第一部分证明创建得到正确公开记录，第二部分证明坏输入没有追加工单、没有消耗下一编号。只检查 422，会遗漏“报了输入错误但仍修改状态”的问题；同时观察下游状态，才保护了本例实际关心的关系。

保存好两个文件并准备相应依赖后，可以运行：

POWERSHELL

```
& .\.venv\Scripts\python.exe -m pytest -q test_mini_api.py
```

这一测试经过了应用接口处理，但没有证明端口 8000 正在监听。前一章的浏览器请求与这里的自动测试各有作用：浏览器让人观察一次真实交换，测试便于重复给定输入并比较精确结果。

完整配套提供 `create_app(...)`，每次调用建立一个新的应用对象、空工单字典和起始编号。因而测试可以各自拿到独立状态，无需共用并清空同一个应用。函数内部的 route、输入模型和工单处理关系，与本章逐步展开的内容对应；它还集中管理了后面摘要服务需要的资源。

哪些值属于请求，哪些值属于运行配置？

创建标题随着每次请求变化，应放在 body。服务显示名称却可能在一次运行期间固定，换一个运行环境时才变化。若只为改变文档标题就编辑代码，运行选择与程序逻辑容易混在一起。

配套使用 Settings 从环境取得这种配置。先看只保留名称的独立片段：

PYTHON
```
from fastapi import FastAPI
from pydantic import Field
from pydantic_settings import BaseSettings


class Settings(BaseSettings):
    app_name: str = Field(
        default="B Ticket API",
        validation_alias="TICKET_API_NAME",
    )


settings = Settings()
app = FastAPI(title=settings.app_name)
```

环境变量 `TICKET_API_NAME` 若存在，`Settings()` 会读取它，把值交给 `app_name`；不存在则使用默认值 `B Ticket API`。`validation_alias` 在这里把外部名字映射到 Python 字段名字。最后是 `FastAPI(title=settings.app_name)` 使用了这个值，文档标题才随之变化。

环境变量不会只凭名字相同就自动改变所有代码。Settings 建立时读取配置，应用创建时使用结果；已经运行的应用也不会因为另一个终端后来改了同名变量就自动更新。

完整配套另外允许代码通过 `Settings(app_name="B Test API")` 提供名称，借助 `populate_by_name=True` 支持字段名输入。它还读取摘要供应商地址和可选凭据：地址决定后续 HTTPX 向哪里发请求，凭据在需要时进入请求的 Authorization header。它们是应用的运行条件，不应让普通工单 body 随意指定。

名称只是普通配置，凭据则需要避免意外显示。配套用 `SecretStr` 让秘密字段在对象的普通显示中呈现掩码，在明确构造授权 header 时才取真实值；这并不自动保护手工打印、截图或终端历史。当前创建和读取不需要任何真实凭据。

直接用 `os.getenv` 读取一两个简单字符串也可以。这里使用 Settings，是因为多项运行配置有默认值、字段名字和需要检查的类型，可以集中描述。它与请求 body model 相似之处是检查输入，不同之处是输入来源和使用时机：一项来自启动环境，一项来自当前 HTTP 请求。

至此，服务端的过程已经可以具体追踪：地址中的目标帮助找到函数，输入声明把消息变成参数，业务函数读取并修改自己的状态，输出声明约束交出的结果。找不到记录、输入不合约、版本不一致和输出构造错误，也各自在不同位置结束正常路线。

接下来回到客户端。它面对的是响应消息，不是服务内部变量；需要亲自判断有没有收到回应、状态能否继续、内容是否可以变成后续代码需要的值。

B03 | 怎样只把可用回应交给后续代码

先只区分三件事：请求有没有发出去、对方回了什么状态、正文是否符合当前合同。

一个 Python 程序现在要取得一段工单摘要。它拥有文字 `"Printer offline"`，另一个服务在 `http://127.0.0.1:8001/summaries` 接收这段文字，成功时回答：

```http
HTTP/1.1 200 OK
Content-Type: application/json

{"summary":"SUMMARY Printer offline"}
```

当前程序真正需要的是其中的字符串 `"SUMMARY Printer offline"`，好把它显示给用户。这里有一段需要程序自己完成的交接：发出请求，拿到回应，检查回应，最后才把字符串交给后面的代码。

这一章从调用方看这段路线。HTTPX 是让 Python 发送 HTTP 请求的库；FastAPI 则是上一章在接收方使用的框架。双方通过 HTTP 约定交接，调用方不用知道接收方的 Python 函数叫什么，接收方也不要求调用方一定使用 HTTPX。

先把一次普通调用读完整

下面是一段用于观察回应的最小调用。运行它需要本机 `8001` 端口已有本章约定的摘要服务；没有电脑或没有启动服务时，先按给出的报文阅读，后面还会介绍不启动服务的受控测试。

```python
import httpx

with httpx.Client(
    base_url="http://127.0.0.1:8001",
    timeout=1.0,
    trust_env=False,
) as client:
    response = client.post(
        "/summaries",
        headers={"X-Request-ID": "R-OK-42"},
        json={"text": "Printer offline"},
    )

    print(response.status_code)
    print(response.headers.get("Content-Type"))
    print(response.text)
```

`httpx.Client(...)` 建立一个客户端对象；`as client` 把这个对象交给名字 `client`。对象可以同时保存配置并提供操作，`client.post(...)` 就是让这个客户端发出一次 POST。点号在这里仍是“取得这个对象提供的某一项”，和上一章用 `payload.title` 取属性有相同的阅读方向。

`with` 划出了客户端的使用区间。进入缩进块以后使用它；离开这个块时，包括中途发生异常时，HTTPX 会关闭它持有的连接资源。先把它理解为“使用完以后有确定的收尾位置”，后面会看到为什么多个请求可以共用这个区间。

几项输入分别落在不同地方：

Python 中写的内容	发出的请求中对应什么
`base_url="http://127.0.0.1:8001"`	这次调用的服务地址
`client.post("/summaries", ...)`	method 为 POST，path 为 `/summaries`
`headers={"X-Request-ID": "R-OK-42"}`	请求头 `X-Request-ID: R-OK-42`
`json={"text": "Printer offline"}`	JSON body `{"text":"Printer offline"}`，并设置 JSON 内容类型

`X-Request-ID` 是双方约定的关联标识。这一章先看它怎样进入 header；下一章用它把两个服务的记录连起来。它不是摘要正文，也不是工单编号。

如果另一个接口约定 query，例如读取 `/tickets/T-42?include_history=true`，HTTPX 可以写：

```python
response = client.get(
    "/tickets/T-42",
    params={"include_history": "true"},
)
```

这是另一条接口的写法示意，`client` 此时必须指向工单 API。`params=` 会把值编码到 URL query；`json=` 会把值编码到 body；`headers=` 写入消息头。它们都是函数调用中的关键字参数，却不是 HTTP 中的同一个位置。位置由服务的接口约定决定，不能为了代码看起来方便就互相替换。

第一段调用还给出 `timeout=1.0`，用来限制等待；它不是整次调用必定在一秒内结束的承诺，稍后会按等待位置拆开。`trust_env=False` 是本地练习的明确选择，让客户端不因当前电脑的代理环境变量而改走代理。

`Response` 到手，还没有到可以使用摘要的时候

正常取得示例回应时，三个 `print` 分别会显示同类结果：

```
200
application/json
{"summary":"SUMMARY Printer offline"}
```

这三个结果来自同一个 `Response` 对象。`response.status_code` 是 HTTP 状态码，`response.headers` 提供消息头，`response.text` 是解码后的 body 文字。`response.text` 显示了大括号，不代表程序已经取得一个字典。

如果再调用：

PYTHON
```
payload = response.json()
```

HTTPX 才会尝试把 body 按 JSON 解析。这个例子解析后得到：

PYTHON
```
{"summary": "SUMMARY Printer offline"}
```

这是 Python `dict`。它和请求发出前的 `{"text": "Printer offline"}` 都在当前客户端进程里，但由不同操作产生，内容和用途也不同。服务器在处理请求时还有它自己的 Python 值；没有任何一个字典对象原样穿过两个进程。

最小观察代码故意只打印了事实。真正要把摘要交给后续程序，还需要回答：状态允许继续吗，内容类型符合约定吗，body 能解析吗，解析结果里面确实有可用的摘要吗？

先检查 status，因为失败回应也可能带着 JSON

供应商维护时可能回答：

HTTP
```
HTTP/1.1 503 Service Unavailable
Content-Type: application/json

{"detail":"maintenance"}
```

这份 body 完全可以解析成 JSON，但它没有提供本次调用需要的摘要。若代码先把所有 JSON 都当结果使用，真正的 HTTP 失败可能在更远的 `payload["summary"]` 处变成一个难以解释的缺键错误。

HTTPX 提供的检查是：

```python
response.raise_for_status()
```

可以把名字按作用读成“如果 status 不成功，就发出异常”。当前 HTTPX 对非 2xx 状态抛出 HTTPStatusError；200、201 等 2xx 会通过。这项检查没有替我们证明“创建接口一定返回201”或“摘要接口一定返回200”。某个接口若要求一个精确状态，还需要明确比较 response.status_code。

在本章摘要客户端中，我们先接受成功类 HTTP 状态，再检查内容。示例服务通常返回200。状态检查成功只允许代码进入下一关，还不能直接 return 摘要。

raise_for_status() 也不会自动替我们处理重定向。HTTPX 默认不跟随重定向，若拿到302，当前检查会停止。只有接口确实需要重定向时，才应进一步核对目标和客户端配置；本地摘要路线没有这个需要。

如果程序只针对某个状态作不同决定，直接写 if response.status_code == 404: 也很清楚。这里所有非成功状态都要停止摘要路线，raise_for_status() 正好把这一选择集中表达出来。

内容类型、JSON 语法和字段，是三件不同的事

现在假定 status 已经通过。客户端与这个摘要服务约定：成功内容的媒体类型是 application/json，JSON 顶层是 object，summary 是非空字符串。

先取得声明的内容类型：

```python
content_type = response.headers.get("Content-Type", "")
media_type = content_type.split(";", 1)[0].strip().lower()
```

headers.get(..., "") 在缺少这个 header 时给出空字符串。下一行串起了几个很小的文字操作：split(";", 1) 最多按一个分号切开，[0] 取前半部分，strip() 去掉两端空白，lower() 转为小写。

例如：

```
Application/JSON; Charset=UTF-8
→ ["Application/JSON", " Charset=UTF-8"]
→ "Application/JSON"
→ "application/json"
```

这样不会因为大小写或附加参数而误拒绝本例允许的内容类型。随后再判断：

```python
if media_type != "application/json":
    raise ValueError("supplier Content-Type is not application/json")
```

`raise ValueError(...)` 主动报告当前内容不能满足这份函数约定；这行运行以后，当前函数的后续语句不会继续。这里先用已熟悉的 `ValueError` 表达坏内容，后面再看配套怎样为它取一个更明确的名称。

这道检查是本程序选择的媒体类型合同。`application/problem+json` 在其他接口中可以有明确用途；本例没有约定它，因此不接收。不能由此推导出其他 JSON 类型都违反 HTTP。

即使 header 是 `text/plain`，body 也可能碰巧写着合法 JSON；通用 JSON parser 仍可能解析成功。那证明的是文字符合 JSON 语法，并没有证明服务遵守了这里的内容类型约定。

反过来，声明正确也不保证实际文字正确：

```http
HTTP/1.1 200 OK
Content-Type: application/json

not-json
```

这个回应通过 status 和媒体类型检查，却会在 `response.json()` 处失败。声明描述了内容应是什么；解析检查的是收到的内容实际能不能按该格式读取。

最后，合法 JSON 不一定带来正确字段：

```json
{"renamed_field":"SUMMARY Printer offline"}
```

或者：

```json
["SUMMARY Printer offline"]
```

前者是字典但没有 `summary`，后者是列表。JSON parser 对两者都可能成功，所以接下来要检查 Python 值：

```python
if not isinstance(payload, dict):
    raise ValueError("supplier JSON is not an object")

summary = payload.get("summary")
if not isinstance(summary, str):
    raise ValueError("supplier summary is not a string")

summary = summary.strip()
if not summary:
    raise ValueError("supplier summary is empty")
```

`isinstance(payload, dict)` 问“这个值是不是字典”。先确定这一点，才调用字典的 `.get()`；缺少 `summary` 时 `.get()` 返回 `None`，下一行就会拒绝它。确定摘要是字符串之后，才能使用 `.strip()`。空字符串在 `if` 判断中为假，因此 `if not summary` 能拦住空内容，也能拦住原本只有空格、经清理后变空的内容。

顺序来自后一项操作对前一项结果的依赖：不能先对数字调用字符串方法，也不能先对列表按字典字段取摘要。

把这些检查接成一个有明确出口的函数

下面是本章使用的完整函数定义。它接收已经建立的 HTTPX Client、正文和关联标识；成功时只返回经过检查的摘要字符串。可以把它与后面的受控测试放在同一个 Python 文件中。

```python
import httpx


def request_summary(client, text, request_id):
    response = client.post(
        "/summaries",
        headers={"X-Request-ID": request_id},
        json={"text": text},
    )
    response.raise_for_status()

    content_type = response.headers.get("Content-Type", "")
    media_type = content_type.split(";", 1)[0].strip().lower()
    if media_type != "application/json":
        raise ValueError("supplier Content-Type is not application/json")

    payload = response.json()
```

PYTHON · 续

```
    if not isinstance(payload, dict):
        raise ValueError("supplier JSON is not an object")

    summary = payload.get("summary")
    if not isinstance(summary, str):
        raise ValueError("supplier summary is not a string")

    summary = summary.strip()
    if not summary:
        raise ValueError("supplier summary is empty")

    return summary
```

这个函数暂时不把所有失败接住。让异常离开函数也是一种明确结果：调用者拿不到成功返回值，并且保留了最先失败的操作。

给定本章开头的成功回应，值会这样变化：

text = "Printer offline"
→ json= 把 {"text": text} 编码为请求 body
→ post 交回 response，status 为200
→ raise_for_status 通过
→ media_type 得到 application/json，通过
→ response.json() 得到 {"summary": "SUMMARY Printer offline"}
→ payload 是 dict，summary 是 str，去空白后仍非空
→ return "SUMMARY Printer offline"

调用位置可以写 `summary = request_summary(client, text, request_id)`。只有右侧函数成功 `return`，左侧赋值才完成；随后才能把这个 `summary` 显示为本次结果。

同样一句“调用失败”，最早停止的位置可能不同

先看没有取得完整回应的情况：

client.post(...) 开始
→ 连接失败，或者某段等待超时
→ post 抛出异常，没有交回完整可用 Response
→ response = ... 的赋值没有完成
→ status、媒体类型和内容检查都没有开始

这里可以读取异常提供的请求信息，却不能访问一个尚未得到的 `response.status_code`。尤其要分清：客户端没拿到完整回应，并不能证明服务端没有收到请求、没有运行函数，或没有完成业务操作。网络把两端分开了；当前代码能直接确认的是自己的调用没有交回完整结果。

再看503：

> post 交回完整 response
> → response.status_code 为503
> → raise_for_status() 抛出 HTTPStatusError
> → 内容检查没有开始

此时回应确实存在。`HTTPStatusError` 的 `.response` 属性保留那份回应，所以错误处理代码可以读 `exc.response.status_code`。这和上一条没有交回 Response 的路线不同。

最后看成功状态下的内容失败：

收到什么	最后通过的检查	最早停止的位置
`200`，`text/plain`，body碰巧是合法JSON	HTTP status	媒体类型比较
`200`，`application/json`，body为`not-json`	媒体类型	`response.json()`
`200`，合法JSON列表	JSON语法	`isinstance(payload, dict)`
`200`，JSON object没有`summary`	JSON顶层类型	字段类型检查
`200`，`summary`为`" "`	字段是字符串	清理空白后的非空检查

这些失败不会产生本次函数的成功返回值。如果某个界面仍保留上一次摘要，那是界面中已有的状态，不能把它当成本次调用成功取得的新结果。

调用者可以按三类边界接住失败。下面是使用 `request_summary` 的调用片段，依赖上面的函数定义和一个已经建立的 `client`：

PYTHON
```
try:
    summary = request_summary(client, "Printer offline", "R-OK-42")
except httpx.TimeoutException:
    print("调用供应商时，某段等待超过了当前边界")
except httpx.RequestError:
    print("没有取得完整可用的供应商回应")
except httpx.HTTPStatusError as exc:
    print(f"供应商返回 HTTP {exc.response.status_code}")
except ValueError:
    print("供应商回应没有提供符合约定的摘要")
else:
    print(summary)
```

`except ... as exc` 把这次异常对象交给名字 `exc`，方便读取它的属性。`else` 属于这个 `try`：只有 `try` 中没有发生异常时才运行，所以失败后不会跌落到显示摘要的成功位置。

`TimeoutException` 是 `RequestError` 的一种。较具体的 timeout 必须写在前面；否则宽泛的 `RequestError` 会先接走它，调用者就失去了“等待超时”这一分类。`HTTPStatusError` 则属于已收到回应之后的状态检查结果。

这里的 `ValueError` 只围绕当前小函数的内容检查使用。程序增长以后，如果把许多无关计算也放进同一个 `try`，那些计算的 `ValueError` 也可能被误称为供应商内容错误。这正是稍后给边界取专用失败名称的原因。

多次调用同一个服务，为什么保留一个 Client

如果只有一次临时请求，`httpx.post(完整URL, ...)` 很直接。程序要反复调用同一个服务时，共用一个 Client 还有更具体的收益：它能复用已建立的连接，并集中保存地址、headers 和等待配置。

建立连接需要工作。Client 内部的连接池保留可以继续使用的连接；下一次请求在条件允许时从池中使用连接，减少重复建立连接的成本。复用的是连接与共同配置，每次 HTTP request 和 response 仍然是独立消息。

例如，下面的使用块依赖前面的 `request_summary` 定义：

```python
with httpx.Client(
    base_url="http://127.0.0.1:8001",
    headers={"Accept": "application/json"},
    timeout=1.0,
    trust_env=False,
) as client:
    first = request_summary(client, "Printer offline", "R-OK-42")
    second = request_summary(client, "Scanner offline", "R-OK-43")
```

两次不同请求使用不同的关联标识。共同的 `Accept` header 表示客户端希望收到JSON；每次请求自己的 `X-Request-ID` 来自函数调用，HTTPX会把这两部分请求头合起来。`Accept` 是希望接收什么，`Content-Type` 是当前这份消息的 body 实际声明成什么，两者方向不同。

离开 `with` 时 Client 关闭。不要把 Client 放在每次循环里面重新建立，却以为仍在复用连接；也不要把它一直留着而没有任何关闭位置。对于命令行程序，`with` 通常可以直接表达使用区间。对于长时间运行的API服务，共享Client可以由应用持有，应用停止时负责关闭。

如果真实服务需要 token，应从运行配置取得，再在构造 `Authorization: Bearer ...` header 的位置使用。token不属于正文或URL，也不应为了检查请求而打印到日志。本章本地练习无需真实秘密值。

一秒 timeout 到底限制哪段等待

HTTPX 区分四个等待位置：

名称	发生在什么时候	到期时的异常
connect	等待建立连接	`ConnectTimeout`
read	等待收到下一块响应数据	`ReadTimeout`
write	等待写出下一块请求数据	`WriteTimeout`
pool	等待从连接池取得可用连接	`PoolTimeout`

这四个异常都属于 `TimeoutException`。一开始用 `timeout=1.0`，是给这些等待设置共同的数值。也可以明确只给连接建立较长时间：

```python
timeout = httpx.Timeout(1.0, connect=2.0)
```

这里其余等待使用1秒，connect使用2秒。它们不是从函数进入时启动的同一只总秒表。连接、写出、读取可能各自消耗时间；读取多个数据块时，每次等待下一块也有自己的边界。因此整个调用的经过时间可能超过1秒。

下一章的慢服务收到请求后先等待1.25秒，期间不发出回应；客户端在等待回应数据超过read边界时停止。这是一个具体的读取超时例子。它不说明所有超时都出在读取，也不说明对方进程被客户端取消了。

超时让当前等待可以结束。结束以后是否再次请求、怎样判断重复操作的业务效果，需要另一组条件；本章只保留明确失败及它的发生位置。

不启动网络服务，也能检查客户端如何面对回应

手工改变真实服务来制造503或坏JSON，需要反复启动、等待。要稳定检查刚才的函数，可以把HTTPX最外侧的传输部分换成受控对象。

transport（传输层）在这里接住已经构造好的 HTTP request，并把传输结果交回。`MockTransport` 允许我们提供一个普通函数作为处理者：HTTPX 调用它时交入 request，它返回指定的 Response，或者主动抛出指定异常。

把下面的代码放在前面的 `request_summary` 定义后面，就能完成一次不访问外网的成功检查：

```python
import json


def reply_with_summary(request):
    assert request.method == "POST"
    assert str(request.url) == "https://supplier.invalid/summaries"
    assert request.headers["X-Request-ID"] == "R-OK-42"
    assert json.loads(request.content) == {"text": "Printer offline"}

    return httpx.Response(
        200,
        json={"summary": "SUMMARY Printer offline"},
    )


transport = httpx.MockTransport(reply_with_summary)
with httpx.Client(
    base_url="https://supplier.invalid",
    transport=transport,
    timeout=1.0,
    trust_env=False,
) as client:
    result = request_summary(client, "Printer offline", "R-OK-42")

assert result == "SUMMARY Printer offline"
```

`assert`后面的条件必须为真，否则这次检查失败。这里不仅检查函数最后返回什么，还检查构造出的method、完整URL、header和body。`request.content`是请求body的bytes；标准库`json.loads`可以把JSON bytes解析成值供比较。

`httpx.MockTransport(reply_with_summary)`传入的是函数本身，没有加调用括号。真正调用这个函数的是HTTPX，它会提供已构造的request。函数里返回的`httpx.Response(200, json=...)`建立一份受控回应，`json=`同时准备JSON内容及对应header。`supplier.invalid`只用于组成目标URL；当前传输已由MockTransport替代，不会连接这个域名。

为了检查读取超时分支，可以换一个处理函数：

```python
def reply_with_timeout(request):
    raise httpx.ReadTimeout("synthetic read timeout", request=request)
```

将它交给httpx.MockTransport后，request_summary会在client.post(...)处抛出ReadTimeout，不会继续到status检查。request=request把当前请求附在异常上，供调用者定位是哪次请求失败。

这项测试没有真的等一秒：它主动提供超时结果，证明代码面对这种结果会走哪条路。下一章先讲清两段交换；要验证真实等待、进程和端口，使用 B05 的双服务运行步骤。

同理，可以让处理函数返回503、返回text/plain、返回坏JSON，分别核对函数停在正确位置。改变一份受控回应，就能在相同请求下比较不同结果；不必让公网是否正常决定这个检查能否重现。

已有的 SupplierClient 包装，究竟增加了什么

现在已经能读懂直接HTTPX调用，再看配套中的SupplierClient就有了具体参照。它把刚才分散的共同配置和检查保存到一个对象中，对外提供：

```python
summary = supplier.summarize(
    text="Printer offline",
    request_id="R-OK-42",
)
```

这里的supplier已经由应用建立。summarize仍然依次发送请求、检查status、检查媒体类型、解析JSON、检查摘要字段，成功返回字符串。包装没有让这些步骤消失。

它还把外部库的失败翻译成本应用使用的名字：

内部发生什么	summarize 对调用者发出什么
httpx.TimeoutException	SupplierTimeout
其他httpx.RequestError	SupplierUnavailable
raise_for_status()得到非成功状态	SupplierBadStatus，保存status code
媒体类型、JSON或摘要字段不符合约定	SupplierBadContent

这些是配套定义的异常类型，可以像ValueError一样被raise发出、被对应except接住。它们的用途是让下一章endpoint只处理“等待超时”“没有完整回应”“坏状态”“坏内容”，不用到处重复HTTPX的细节。

例如以下是配套发送部分的核心摘录，位于`SupplierClient.summarize`方法内，`self._client`是当前supplier对象保存的HTTPX Client：

PYTHON
```
try:
    response = self._client.post(
        "/summaries",
        headers={"X-Request-ID": request_id},
        json={"text": text},
    )
except httpx.TimeoutException as exc:
    raise SupplierTimeout("supplier timed out") from exc
except httpx.RequestError as exc:
    raise SupplierUnavailable("supplier produced no response") from exc
```

`raise ... from exc`在对外给出本应用失败名称的同时，保留原始异常作为原因。它没有恢复成功，也不会从异常位置继续往下运行。查看错误追踪信息时，仍可以知道底层究竟是ConnectTimeout还是ReadTimeout。

这种包装叫 adapter（适配层）：把一侧的调用方式和结果翻译为另一侧使用的接口。如果整个脚本只有一处临时 HTTP 调用，直接用 HTTPX 可能更清楚；当多个 endpoint 都要调用同一供应商、统一检查内容并统一解释失败时，这一层就有了具体作用。

SupplierClient长期保存Client，因此也必须有收尾负责人。配套应用自己建立它，就在应用停止时调用`close()`；测试传入的替身由提供它的一方管理。下一章使用它时，关注的交接已缩小为：提交文字与request_id，取得摘要字符串，或者得到一种明确失败。

到下一章，刚才的客户端不再直接面对用户。它会放进FastAPI服务内部：用户先请求本地API，本地API再请求摘要供应商。我们要把两次HTTP回应分别认清，再用关联标识还原一次处理的经过。

B04 | 两次 HTTP 交换怎样找到失败位置

先沿一个 request_id 看 API 和合作方的两次交换，再判断最早哪一边没有交出可用结果。

本地工单API已经能创建工单。现在它增加一项功能：调用者提交一段工单文字，API向摘要供应商取得摘要，再把公开结果交回调用者。

为了固定背景，先假定本地API的内存中已有这一条记录：

```JSON
{"id":"T-42","title":"Printer offline","priority":2,"status":"open","version":
  1}
```

这条记录属于当前工单API进程。摘要供应商不保存这份工单表，也不知道本地的版本号；它只接收本次提交的文字并生成摘要。这里摘要的是请求body里的`text`，并不是API自动从已存工单中读取所有内容。`T-42`用于确认这次摘要请求指向一条存在的本地工单。

本地 API 既接收请求，也会发出请求

调用者向工单API发送：

```HTTP
POST /tickets/T-42/summary HTTP/1.1
Host: 127.0.0.1:8000
X-Request-ID: R-OK-42
Content-Type: application/json

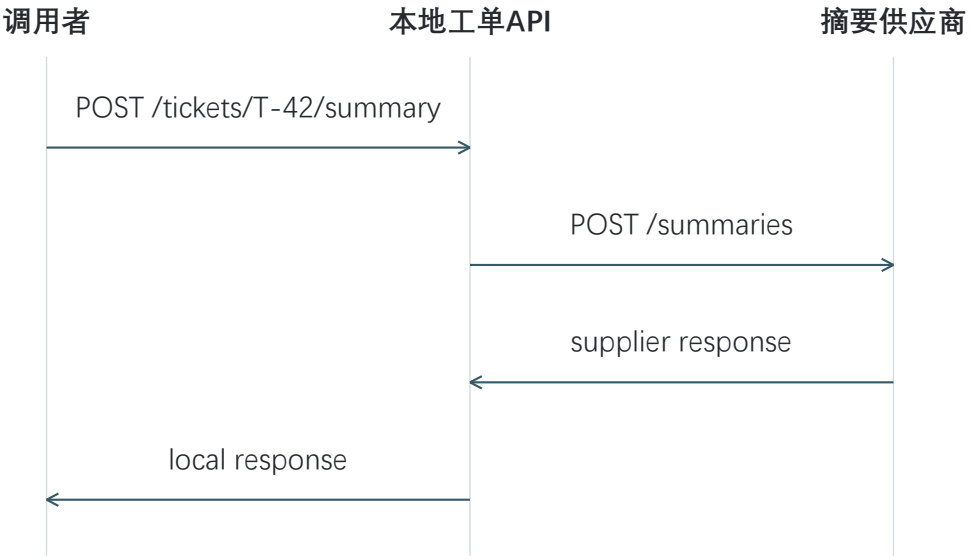
{"text":"Printer offline"}
```

这只是第一段交换的开始。本地API还要向`127.0.0.1:8001`上的摘要服务发送另一份请求：

```HTTP
POST /summaries HTTP/1.1
Host: 127.0.0.1:8001
X-Request-ID: R-OK-42
Accept: application/json
Content-Type: application/json

{"text":"Printer offline"}
```

把它们放在一条执行路线上：



本地API在第一段中是server：它接收请求并回答。在第二段中，它通过HTTPX成为client：它请求供应商并检查回应。client和server描述的是一次交互中的职责；同一个程序可以在不同交互中承担两种职责。

最初调用者看见的Response来自本地API。供应商返回的另一份Response先交给本地API内部的HTTPX客户端。即使两份回应都为200，它们也有不同的接收者和不同的内容合同。

本章报文为阅读方便省略工具自动生成的长度等header。`8000`和`8001`用于说明各自服务入口；实际验证入口可以选择另外两个空闲的本机端口，但请求必须分别指向对应服务。

两条请求交错时，怎样知道一行日志属于哪次工作

先看没有关联标识的日志：

```
supplier_call_started
supplier_call_started
supplier_bad_status
supplier_call_succeeded
```

如果两个调用者几乎同时请求摘要，这些事件可能交错出现。第一条started究竟对应成功还是失败，光靠行的位置不一定能回答。

给每次外部请求一个关联标识，关系就能看见：

```JSON
{"event":"supplier_call_started","request_id":"R-OK-42","ticket_id":"T-42"}
{"event":"supplier_call_started","request_id":"R-FAIL-43","ticket_id":"T-42"}
{"event":"supplier_bad_status","request_id":"R-FAIL-43","supplier_status":500,
  "status":502}
{"event":"supplier_call_succeeded","request_id":"R-OK-42","status":200}
```

现在可以把同一个 `request_id` 的事件取在一起，分别还原两次处理。correlation（关联）在这里做的就是这件事：把分散的记录对应回同一次工作。

同一次外部请求经过本地API与供应商时沿用一个标识；下一次新的外部请求使用新的标识。本章采用：

本次外部请求	request_id	目标业务工单
正常摘要	`R-OK-42`	`T-42`
让假供应商产生500	`R-FAIL-43`	`T-42`
让本地客户端等待供应商时超时	`R-SLOW-44`	`T-42`
提交空文字	`R-BAD-45`	`T-42`

多次处理可以指向同一张工单，但不能因此共用一次处理的身份。`T-42`回答“操作哪条业务记录”；request_id回答“这一组请求和日志属于哪次处理”。request_id不是业务主键，也不是防止重复写入的幂等键。相同request_id不会自动让服务器识别重复业务、更不会自动避免第二次产生效果。

本章让调用者明确提供这些合成标识，便于阅读。应用也可以在缺少标识时生成新的值，然后把它传到下游并放回response header。生成ID并不证明业务成功；它只是为之后关联成功或失败证据提供名字。

Header 怎样进入函数，再进入下一次请求

FastAPI endpoint可以这样声明请求头参数：

```PYTHON
x_request_id: str | None = Header(default=None)
```

这是一段函数参数声明，`Header`由`fastapi`导入。`str | None`表示值可以是字符串，也可以缺失为`None`；默认值放在`Header(default=None)`里。FastAPI默认把参数名中的下划线转换成header名中的连字符，因此`x_request_id`对应`X-Request-ID`。

当调用者发送X-Request-ID: R-OK-42时，函数看到的x_request_id就是字符串"R-OK-42"。如果要兼容缺失或空白值，最小的普通Python写法可以这样读：

PYTHON
```
from uuid import uuid4


def choose_request_id(value):
    if value is not None:
        cleaned = value.strip()
        if cleaned:
            return cleaned
    return f"R-{uuid4().hex}"
```

这是一个完整的小函数：存在可用文字就返回它，否则生成新标识。uuid4()产生随机UUID对象，.hex取其十六进制文字；它只是避免每次都手工起名。日志关联仍要求各次请求不要故意复用同一个值。本章给定的R-OK-42会直接经过第一条返回路线。

沿着一次请求，本地标识会分别交给这些位置：

```
调用者 request header
→ FastAPI的x_request_id参数
→ 本地request_id变量
  ├── 本地日志字段
  ├── supplier.summarize(..., request_id=request_id)
  │   → HTTPX的headers={"X-Request-ID": request_id}
  │   → 供应商收到的request header
  └── 本地最终response header
```

本地响应头使用自己的 request_id，不依赖供应商返回这个值；后面分别看正常返回与异常路径怎样把它交给调用者。这些分支没有改变正文，也没有替正文做校验，而是用同一个标识把本章的控制流程和多处证据连在一起。

本地 API 先决定输入能否继续

摘要请求的输入与公开输出使用两份 model。下面是完整的声明，可以和 B02 的工单路由放在同一个应用中：

```python
from pydantic import BaseModel, Field


class SummaryInput(BaseModel):
    text: str = Field(min_length=3, max_length=500)


class SummaryPublic(BaseModel):
    ticket_id: str
    summary: str
    request_id: str
```

`SummaryInput`要求text是字符串，长度在3到500之间。model接住JSON字段后，endpoint通过`payload.text`取得校验后的值。`SummaryPublic`约定对外公开工单编号、摘要和关联标识，不把HTTPX的Response对象或供应商内部报错原样送出去。

先把同步endpoint最关键的成功部分读出来。下面是服务内部的核心摘录：`app`已经是FastAPI应用，`app.state.tickets`已有本章开头的T-42，`app.state.supplier`是B03介绍的、由应用持有的SupplierClient。`choose_request_id`是上面的小函数。这里暂时让供应商异常继续向外抛出，稍后再补本地失败回应。

```python
from fastapi import Header, HTTPException, Response


@app.post("/tickets/{ticket_id}/summary", response_model=SummaryPublic)
def summarize_ticket(
    ticket_id: str,
    payload: SummaryInput,
    response: Response,
    x_request_id: str | None = Header(default=None),
):
    request_id = choose_request_id(x_request_id)
    response.headers["X-Request-ID"] = request_id

    if ticket_id not in app.state.tickets:
```

PYTHON · 续

```
        raise HTTPException(
            status_code=404,
            detail="ticket_not_found",
            headers={"X-Request-ID": request_id},
        )

    summary = app.state.supplier.summarize(
        text=payload.text,
        request_id=request_id,
    )

    return {
        "ticket_id": ticket_id,
        "summary": summary,
        "request_id": request_id,
    }
```

`ticket_id`来自path，`payload`来自body，`x_request_id`来自header。`response: Response`则由框架提供，是让函数增加回应header的对象，不是调用者在body中提交的字段。

`.state`是应用保存自己所持有值的位置。这里放了内存工单表和supplier对象；这种保存方式没有建立数据库。停止并重新创建应用后，工单状态不会因此自动保留。

查找T-42发生在调用supplier以前。找不到目标时，`raise HTTPException`停止函数；供应商不会被调用。存在目标时，函数才把`payload.text`和request_id传给supplier。supplier成功交回字符串之后，才形成公开dict。

`response.headers[...]`设置普通成功回应上的header。抛出`HTTPException`时框架会形成另一份失败回应，因此错误分支还要通过`headers=...`明确带回request_id，不能只依赖先前修改的成功Response对象。

成功时，有两份回应和一次本地返回

给定`R-OK-42`和`{"text":"Printer offline"}`，成功路线按真实执行顺序展开：

> 本地route匹配 /tickets/T-42/summary
> → SummaryInput通过，payload.text = "Printer offline"
> → endpoint开始，request_id = "R-OK-42"
> → 在本地内存表找到T-42
> → supplier.summarize收到文字与R-OK-42
> → HTTPX发出 POST /summaries
> → supplier返回200、application/json和摘要JSON
> → adapter检查status、媒体类型、JSON、summary字段
> → adapter return "SUMMARY Printer offline"
> → endpoint return公开dict
> → SummaryPublic检查公开输出
> → 本地API向最初调用者返回200

供应商的回应是：

```http
HTTP/1.1 200 OK
Content-Type: application/json

{"summary":"SUMMARY Printer offline"}
```

最初调用者得到的回应则是：

```http
HTTP/1.1 200 OK
X-Request-ID: R-OK-42
Content-Type: application/json

{"ticket_id":"T-42","summary":"SUMMARY Printer offline","request_id":"R-OK-42"}
```

中间的`return summary`是Python函数内部交接，最后的HTTP response是跨进程交接。两者分别发生，才能解释本地API为什么会在供应商结果上增加ticket_id和request_id。

本章使用的本地假供应商按固定规则给输入加`SUMMARY `前缀，因此`Printer offline`对应`SUMMARY Printer offline`。它没有真的理解工单内容；这个可预测的结果让我们能集中验证两次HTTP交接。

日志记录阶段，使处理经过可以被读回

为了之后区分“准备调用”“成功取得结果”和“依赖失败”，可以在这些位置记录小型事件：

> 调用supplier以前 → supplier_call_started
> 取得受检摘要以后 → supplier_call_succeeded
> 接住坏status以后 → supplier_bad_status
> 接住timeout以后 → supplier_timeout

一个写事件的辅助函数可以是：

```python
import json
import logging

logger = logging.getLogger("ticket_api")


def emit_event(event, request_id, **fields):
    entry = {"event": event, "request_id": request_id}
    entry.update(fields)
    logger.info(json.dumps(entry, ensure_ascii=False))
```

这是日志内容形成方式的完整小例，应用还需要配置INFO级别和输出位置，事件才会显示。Python日志的默认门槛不会自动把INFO都打印出来；随书服务在启动配置中明确建立INFO输出。

参数中的`**fields`把额外的关键字参数收成一个字典。调用`emit_event("supplier_bad_status", request_id, supplier_status=500, status=502)`时，`fields`就包含后面两个字段。`entry.update(fields)`把它们加入当前事件；`json.dumps`把字典变成一行JSON文字。

这些代码并不会自动判断哪些字段安全。当前调用位置只传入阶段、标识、工单编号和状态码，不传正文或token。日志的任务是说明执行走到哪里，不需要复制工单内容才能把同一次工作关联起来。

`supplier_call_succeeded`也要读准范围：它表示本地adapter已经取得符合合同的摘要，事件发生在endpoint最终输出以前。如果之后输出model校验失败，这行日志不能单独证明最初调用者已经拿到了200。

空文字为什么只能得到一段 HTTP 交换

现在换成另一份外部请求：

```HTTP
POST /tickets/T-42/summary HTTP/1.1
Host: 127.0.0.1:8000
X-Request-ID: R-BAD-45
Content-Type: application/json

{"text":""}
```

`text`长度为0，SummaryInput要求至少3个字符。结果是：

```
请求到达本地FastAPI
→ route匹配
→ body可以解析为JSON
→ SummaryInput长度校验失败
→ 422 response
```

`summarize_ticket`没有开始运行，所以它内部的`choose_request_id`、工单查找和supplier调用都未执行。这里仍有第一段的request / response；第二段request根本没有发生。

测试这个分支时，仅检查422还不够表达本章关心的关系。测试还应让supplier替身记录调用次数，然后检查它为零。若应用之前已经有T-42，失败后T-42仍存在；不能把“本次没有写入”写成“所有内存状态必须为空”。

还有一个实际问题：endpoint没有执行，谁把R-BAD-45放进失败日志和response header？这需要应用在框架处理输入错误的位置统一加上标识。

框架会把收到的Request和校验异常交给已登记的错误处理函数。该函数从Request header取出ID，记录`validation_failed`，交回框架原有的422回应，再给这份回应加上`X-Request-ID`。它的输入输出可以完整写成：

```
输入：原request，header含R-BAD-45；RequestValidationError
→ 取出R-BAD-45
→ 记录 event=validation_failed，request_id=R-BAD-45，status=422
→ 使用FastAPI默认的输入错误回应
→ 在该回应header加入X-Request-ID: R-BAD-45
输出：422 response
```

这一处理位置在endpoint外面，因此能覆盖endpoint尚未运行的情况。配套应用提供了这层支持。理解本条失败路线，需要知道谁接住了错误、用到什么输入、返回什么；不需要把整个应用的异步启动协议一起引入。

供应商已经返回500，本地为什么选择回答502

本地假供应商约定：收到合成文字"supplier-500"时返回500。调用者用新的R-FAIL-43发出：

```http
POST /tickets/T-42/summary HTTP/1.1
Host: 127.0.0.1:8000
X-Request-ID: R-FAIL-43
Content-Type: application/json

{"text":"supplier-500"}
```

这段文字长度合法，所以本地输入校验通过；T-42也存在。第二次request发出以后，供应商确实回答500。B03的adapter在raise_for_status()处发现失败，抛出SupplierBadStatus，其中保存status_code=500。

本地endpoint可以接住这一失败，形成自己的公开回应。下面这段替换前面endpoint中直接调用supplier的部分；其他参数、model与返回dict沿用前面的上下文：

```python
emit_event("supplier_call_started", request_id, ticket_id=ticket_id)
try:
    summary = app.state.supplier.summarize(
        text=payload.text,
        request_id=request_id,
    )
except SupplierBadStatus as exc:
    emit_event(
        "supplier_bad_status",
        request_id,
        supplier_status=exc.status_code,
        status=502,
    )
    raise HTTPException(
        status_code=502,
        detail="supplier_bad_status",
        headers={"X-Request-ID": request_id},
    ) from exc

emit_event("supplier_call_succeeded", request_id, status=200)
```

SupplierBadStatus需要从配套的supplier_client模块导入。except ... as exc取得本次adapter异常，exc.status_code是它保留的供应商状态。raise HTTPException让本地API为最初调用者形成502；from exc保留这次公开失败的内部原因。

现在两份回应各归其位：

> 供应商 → 本地API：500
> 本地API → 最初调用者：502，detail为supplier_bad_status

502表示这个中间服务没有取得可用的上游结果。在这里选择502，是本地API的公开合同；HTTPX只报告供应商状态，没有替本地API决定最终响应码。程序也没有把供应商原始response的数字原地改成502，它建立的是第一段交换自己的回应。

`supplier_call_started`之后的日志会是：

```JSON
{"event":"supplier_bad_status","request_id":"R-FAIL-43","supplier_status":500,
  "status":502}
```

这里同时保存两个状态，是因为它们分别属于不同交换。若只记录最终502，后来就无法仅凭这行记录知道原始供应商究竟返回500，还是返回了成功状态却缺少摘要。

超时路线缺的是供应商完整回应，本地仍然可以回答504

另一次外部请求使用`R-SLOW-44`，body为：

```JSON
{"text":"supplier-slow"}
```

假供应商对这段合成文字先等待1.25秒，再准备正常摘要；等待期间不发出回应。当前adapter的HTTPX配置把read等待边界设为1秒。因此本条路线是：

> 本地输入校验通过，T-42存在
> → 第二次request开始
> → 本地HTTPX等待供应商回应数据
> → 该段读取等待超过1秒
> → ReadTimeout，post没有交回完整可用Response
> → adapter抛出SupplierTimeout
> → endpoint形成自己的504 response

在同一个try后面增加这个分支即可表达本地选择：

```python
except SupplierTimeout as exc:
    emit_event("supplier_timeout", request_id, status=504)
    raise HTTPException(
        status_code=504,
        detail="supplier_timeout",
        headers={"X-Request-ID": request_id},
    ) from exc
```

这是接在前述supplier调用try后的异常分支摘录，SupplierTimeout同样由supplier_client导入。每次异常只进入与它匹配的一个分支，执行raise以后不会继续到成功日志或成功返回。

调用者现在看见：

```http
HTTP/1.1 504 Gateway Timeout
X-Request-ID: R-SLOW-44
Content-Type: application/json

{"detail":"supplier_timeout"}
```

“没拿到response”在这里必须带上观察位置：本地API内部的HTTPX没有拿到完整可用的供应商Response；最初调用者却拿到了本地API形成的504。

不能据此推出供应商没有执行。它可能已经接收文字，甚至在客户端停止等待以后继续完成处理。timeout结束的是当前客户端某段等待，并不自动撤销另一个进程的工作。本章没有在超时后加入再次请求。

1秒也不是整条“调用者→本地API→供应商→调用者”路线的总时限。连接、写出、读取等阶段各有等待边界；如果需要整个业务流程的总时间限制，那是另一个明确的控制要求。

连接失败和坏内容也要保留自己的原因

如果摘要服务没有监听相应端口，HTTPX可能得到ConnectError。这次没有取得完整可用的供应商回应，但它不是等待到期；adapter会给出SupplierUnavailable。本地API选择：

```
SupplierUnavailable
→ event=supplier_unavailable
→ 本地503
→ detail=supplier_unavailable
```

如果供应商返回200，却没有符合合同的摘要，则是另一条路线：

> 供应商response已到手
> → status通过
> → 媒体类型、JSON或summary字段检查失败
> → SupplierBadContent
> → event=supplier_bad_content
> → 本地502
> → detail=supplier_bad_content

这两个分支采用和前面相同的写法：接住对应异常，记录事件，`raise HTTPException`，附回request_id。并列起来，本地失败合同是：

最早失败的位置	本地给调用者的结果	supplier是否已被本地调用
本地SummaryInput拒绝空文字	422	否
本地查不到目标T-42	404，`ticket_not_found`	否
非timeout的发送 / 连接失败	503，`supplier_unavailable`	已尝试调用；不代表供应商收到
某段等待超时	504，`supplier_timeout`	已尝试调用；服务端效果不能由超时确定
收到供应商非成功status	502，`supplier_bad_status`	是，并已取得回应
成功status后的内容不合约	502，`supplier_bad_content`	是，并已取得回应

一个status有时对应不止一个内部原因，例如502既可能是坏状态，也可能是坏内容。公开的detail和同request_id的日志保留这种区别。也不要把表格推广成所有API的固定规则；它描述的是这份应用的约定。

最初调用者的 Network 能看到哪一段

在浏览器中调用本地API时，Network记录的是浏览器发出的请求与它收到的回应。浏览器可以看到`POST /tickets/T-42/summary`、R-FAIL-43以及本地502，但不会因此自动多出本地Python进程发给供应商的那条HTTP记录。

因此看到502以后，下一步是带着request_id查看本地记录：

> Network：R-FAIL-43，本地502
> → 本地日志：supplier_call_started，R-FAIL-43
> → 本地日志：supplier_bad_status，supplier_status=500，R-FAIL-43
> → 定位：第二次交换已有response，在status检查处停止

如果本地日志是`supplier_bad_content`，调查应转向B03的内容检查。若是R-BAD-45的422，并且测试显示supplier零调用，就先看本地输入约束；供应商没有日志是符合这条路线的结果，不能把“没被调用”诊断为“供应商崩溃”。

日志应当如实标记观察到的阶段。`supplier_call_started`证明本地程序进入了调用位置，不证明对方已经收到；供应商自己记录的`supplier_request_received`才补充了接收方的证据。把这种差别说清，才不会把一条“准备做”的记录误读成“已经完成”。

先在函数交接处固定结果，再经过两个真实服务

要检查endpoint如何处理supplier结果，可以先提供一个可记录调用的替身。它履行相同的小接口：`summarize(text=..., request_id=...)`，成功返回字符串，也可以抛出指定失败。

下面是这个替身的完整定义：

PYTHON
```
class RecordingSupplier:
    def __init__(self):
        self.calls = []

    def summarize(self, *, text, request_id):
        self.calls.append({"text": text, "request_id": request_id})
        return "SUMMARY Printer offline"
```

`RecordingSupplier()`建立一个对象时，Python调用`__init__`完成初始准备；`self`指向正在准备或使用的那个对象。`self.calls = []`为它保存一份空列表。以后调用同一个对象的`summarize`，它会向这份列表追加一条记录，再返回固定摘要。

方法参数中的单独*要求后面的text和request_id用名字传入，因此调用处清楚写出哪个值属于哪个输入。这里没有新的网络行为；这个类只把“是否调用过、传了什么”保留下来。

测试把该对象交给应用的`create_app(..., supplier=supplier)`入口。应用把它存到`app.state.supplier`，endpoint仍照常调用`summarize`，只是接收调用的对象变成了测试替身。

成功请求之后，可以看到：

```
PYTHON
[{"text": "Printer offline", "request_id": "R-OK-42"}]
```

这是一项调用的列表。另建一份新的替身和测试应用，再发空文字请求，替身的列表应保持为空，因为 endpoint 尚未运行。如果在先前已成功调用的同一个替身上接着发坏请求，应检查列表没有新增，而不是要求原有记录消失。若要测试 500 映射，就让替身在记录调用后抛出 `SupplierBadStatus(500)`；让它抛出 `SupplierTimeout`，可以核对本地是否形成 504。

这些检查回答的是endpoint与adapter之间的函数交接。B03的MockTransport替换得更靠外：它让真正的adapter面对受控HTTP Response，以验证status和内容检查。两种替身放在不同位置：

```
TestClient → FastAPI endpoint → 记录型supplier替身
                    证明调用参数、零调用、公开错误映射

真实SupplierClient → HTTPX → MockTransport
                    证明HTTP请求构造、status与内容检查
```

两种测试都没有建立第二个真实网络服务。它们通过之后，再启动独立的本地工单API和假供应商，让请求穿过两个监听端口，才补上进程间交换的证据。

真实双服务验证要把不同请求分别认出来

一次受控验证先创建工单，取得实际返回的id，再按这个id依次请求摘要。新应用的首次创建可以得到T-42；如果复用已经创建过记录的应用，应使用实际id，不能因编号不同就误判接口失败。

每条外部请求都使用自己的标识，并在两段HTTP间沿用：

> R-OK-42
> 调用者 → 本地API → 假供应商 → 本地API → 调用者
> 预期：受检摘要，本地200
>
> R-FAIL-43
> 调用者 → 本地API → 假供应商500 → 本地API → 调用者
> 预期：本地502，日志保留supplier_status=500
>
> R-SLOW-44
> 调用者 → 本地API → 假供应商等待
> 本地HTTPX读取超时 → 本地API → 调用者
> 预期：本地504，日志为supplier_timeout

R-BAD-45则只到本地422，第二段没有发生。验证时不仅比较最终status，也要检查每次response header返回对应ID，API和供应商在实际进入的阶段记录对应ID，失败路线没有落入成功结果显示。

两个进程都应由这次本地验证明确启动，并在结束时关闭；监听端口和运行输入由验证入口给出。这样得到的结果说明这套受控本机路线确实经过了HTTP连接。它不能自动扩大成公网供应商或客户环境已通过，正如替身测试通过也不能替代这次真实调用。

如果最后只记得“摘要失败了”，仍会在几层代码之间猜测。能恢复的描述应该具体到一次处理：R-FAIL-43已通过本地输入校验并取得供应商500，本地adapter在status检查处停止，endpoint把它形成自己的502。这个描述同时指出了最后成功的位置、第一处不符合约定的位置，以及最初调用者看见的结果。

下一章把创建、查询、客户端检查和这里的关联证据接成一个可以独立说明的小型API里程碑。工单仍保存在本地服务内存中；这一步先证明跨进程交接能够解释、测试和定位。

B05 | 创建之后，怎样读回同一条记录？

先用同一个返回 ID 创建并读回；启动成功只是入口，业务结果和证据还要分别核对。

一个接口能在文档页面里显示出来，还不等于两个程序已经一起完成工作。这一章把创建与查询连接成一项具体任务：客户端提交标题和优先级，服务建立工单，把编号交回；客户端取得这个编号，再向服务查询。

这次最重要的连接不是再增加一条 route，而是：**第一次回应中的编号，决定第二次请求查谁。** 编号必须从回应里取，不能因为例子里常见 `T-42`，就在客户端里一直写死它。

两次交换怎样连成一件事？

任务从这份输入开始：

```python
title = "Printer offline"
priority = 2
```

服务刚启动且没有创建过记录时，第一条工单编号是 `T-42`。创建与查询的关键消息如下；这是便于阅读的报文节选，不列连接等自动生成的字段。

```http
POST /tickets HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{"title":"Printer offline","priority":2}
```

```http
HTTP/1.1 201 Created
Content-Type: application/json

{"id":"T-42","title":"Printer offline","priority":2,"status":"open","version":
  1}
```

客户端把回应解析成一个新的 dict，取出 `created["id"]`，得到字符串 `"T-42"`。它用这个值组成查询地址：

```http
GET /tickets/T-42 HTTP/1.1
Host: 127.0.0.1:8000
```

服务返回 200，并返回同一份公开记录。没有人在两次交换之间修改记录时，查询得到的五个公开字段应与创建结果相同。

> 标题与优先级
> → 创建请求
> → 服务建立记录
> → 创建回应给出 id
> → 客户端取出这个 id
> → 把 id 放进查询请求的 path
> → 服务按 id 找回记录

如果再次运行创建任务，服务可能给出 T-43。这不是程序出错；它说明服务的内存状态已改变。只要查询使用本次返回的编号，连接就仍然成立。这里先用单个客户端顺序操作；这份内存示例不是多用户并发写入的生产实现。

在电脑上，先把项目放到哪里？

从资料包根目录的可见入口 01_配套代码_A-E 进入 examples → b_http_api → ticket-api。这个入口与下面的底层相对路径指向同一份配套文件：

> 01_配套练习/examples/b_http_api/ticket-api

用文件资源管理器把整个 ticket-api 文件夹复制到自己选择的新实验目录。保留原始随书目录不动；下面的环境、测试和新增脚本，都在实验副本里进行。如果目标已有自己的修改，另选一个新目录，不覆盖旧实验。

在复制出来的 ticket-api 文件夹打开 PowerShell。先看当前位置和文件，确认这里确实有 app.py、client.py 与 requirements.txt：

```powershell
Get-Location
Get-ChildItem
```

这个项目的几份文件分别连接不同的位置：

文件	需要找它时，正在处理什么问题
app.py	服务怎样接入请求、检查输入、创建与查询记录
client.py	调用方怎样发送请求并检查工单回应

文件	需要找它时，正在处理什么问题
`supplier_client.py`	工单服务怎样调用摘要服务并区分失败
`fake_supplier.py`	怎样在本机稳定得到成功、500 或慢响应
`test_app.py`、`test_client.py`、`test_supplier_client.py`	修改后，哪些已知行为需要重新比较
`requirements.txt`	这个项目要使用哪些依赖版本

B02 的 `mini_api.py` 是用来逐步看懂一条 route 的独立小例；这里运行的是随书的完整 `app.py`。完整项目还把配置、日志和共享 HTTPX Client 的关闭集中管理。它的 `create_app()` 每次建立一份应用和独立内存状态，便于测试使用新的起点；不是把几章的小例机械拼到同一个文件。

怎样准备同一个 Python 环境？

先确认准备使用的解释器：

```powershell
python --version
```

配套要求 Python 3.10 或更高版本。如果命令不存在，或显示的版本低于要求，先按 A09 的解释器与环境定位方法检查；需要安装时，可查 Python 官方 Windows 使用说明。确认调用到适用的解释器后，再回这里建立环境，不要在尚未确定的 Python 上继续安装依赖。

下面用它在实验目录建立 `.venv`。后面的安装、测试和启动，都明确调用这个虚拟环境里的 Python，避免“包装在一个环境，服务用另一个环境”。

```powershell
python -m venv .venv
```

这条命令成功后，再安装项目列出的版本：

```powershell
& .\.venv\Scripts\python.exe -m pip install -r .\requirements.txt
```

安装完成，再分别检查依赖和现有测试：

```powershell
& .\.venv\Scripts\python.exe -m pip check
```

```powershell
& .\.venv\Scripts\python.exe -m pytest -q
```

如果某一步出现错误，停在那一步处理，不继续启动服务。`pip check` 回答已装依赖之间是否兼容；测试回答给定输入下的行为是否符合断言。两者都需要，但不是同一个问题。版本文件固定了直接依赖，并不代表所有电脑、所有 Python 版本都已验证过；依赖安装与测试结果仍以当前实验环境为准。

创建虚拟环境、按依赖文件安装、用同一个解释器运行，应当在后面的项目中逐渐用顺。没必要把库的版本号背下来；版本号放在文件里，执行时核对它。

为什么要打开两个窗口？

第一个窗口持续运行服务，等待请求；第二个窗口运行主动发送请求的客户端。它们是两个进程，不是同一段代码里两个函数的先后调用。

先在第一个窗口查看端口 `8000` 有没有被别的进程占用：

```powershell
Get-NetTCPConnection -State Listen -ErrorAction Stop |
    Where-Object LocalPort -eq 8000
```

若显示监听记录，先确认它是谁，不要继续启动，也不要随意结束不认识的进程。查询报错时，不能把失败当成端口空闲；先处理查询问题。只有查询成功且没有匹配记录时，再运行下面的启动命令。检查与启动之间仍可能有其他程序占用端口，所以随后还要看启动结果：

```powershell
& .\.venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000
```

这里有两个同名但位置不同的 `app`：冒号前面的 `app` 是 Python 模块 `app.py`；冒号后面是该模块里的应用对象 `app`。Uvicorn 加载这个对象并监听 `127.0.0.1:8000`。FastAPI 随后负责 route 匹配、校验和函数交接。只绑定 `127.0.0.1`，是为了让本次练习从本机访问，不公开到局域网。

看到新启动的进程明确显示运行地址后，保持这个窗口开着。如果命令立即报错并回到提示符，说明这次服务没有正常持续运行，不能把浏览器碰巧看到的别的页面当成启动成功。

现在打开第二个 PowerShell 窗口，也进入同一实验副本。两个窗口的当前位置需要分别确认，窗口不会自动共享当前位置的后续变化。

用本次返回的编号，再发一次查询

在实验副本里新建 UTF-8 纯文本文件 `check_exchange.py`，保存下面的完整代码：

```python
import httpx

from client import create_ticket, get_ticket


with httpx.Client(
    base_url="http://127.0.0.1:8000",
    timeout=2.0,
    trust_env=False,
) as client:
    created = create_ticket(
        client,
        title="Printer offline",
        priority=2,
    )
    ticket_id = created["id"]
    fetched = get_ticket(client, ticket_id)

    assert fetched == created
    print("created:", created)
    print("fetched:", fetched)
    print("same public record:", ticket_id)
```

`from client import ...` 从同目录的 `client.py` 取出两个已有函数。这里的模块名 `client` 和 `with` 后保存 HTTPX 对象的变量名恰好相同；两个函数已经被导入为独立名称，不会因此变成别的函数。`create_ticket` 接收标题与优先级，完成创建请求与回应检查；`get_ticket` 接收编号，完成查询。

`trust_env=False` 让这次本机调用不自动采用环境中的 HTTP 代理设置。`timeout=2.0` 限制各类网络等待，不是“整件创建加查询工作只能花两秒”。`with` 结束时会关闭这个 HTTPX Client；它不会关闭另一个窗口的服务。

正文里真正把两次交换连起来的是这两行：

```python
ticket_id = created["id"]
fetched = get_ticket(client, ticket_id)
```

第一行不是新建工单，只是读取刚才得到的结果。第二行才发出另一条 HTTP request。后面的 `assert fetched == created` 把本次查询结果与本次创建结果比较；不同就抛出 `AssertionError`，不会打印最后的成功说明。这个断言用于实验验证，不作为对外接口的输入校验。

在第二个窗口运行：

```powershell
& .\.venv\Scripts\python.exe .\check_exchange.py
```

若从一个新服务开始，最后一行应是：

```
same public record: T-42
```

前两行分别显示创建与查询的 dict。回到服务窗口，访问日志中应能找到本次 `POST /tickets` 的 `201` 和随后 `GET /tickets/T-42` 的 `200`。脚本比较两份公开结果；访问日志补上各次请求的状态证据。不能只凭客户端退出码为零，就推断所有具体 HTTP 状态已经被脚本严格断言。

如果出现 `ApiTransportError`，先确认当前服务和地址；`ApiStatusError` 表示已有非成功回应，查看状态及错误原因；`ApiContentError` 表示回应内容不符合客户端的小合同。失败时这份检查脚本让异常直接显示出来，方便实验定位；随书 `client.py` 的命令行入口则会捕获这些失败并显示 `FAILED ...`。两种呈现方式不改变失败发生的位置。

为什么服务检查过了，客户端还要检查？

在这个项目中，`client.py` 先检查状态，再要求回应声明精确的 `application/json` 媒体类型，随后解析 JSON object，最后核对工单的五个字段及类型。它不只是执行 `response.json()` 就放心继续。

原因是客户端面对的是它实际收到的回应，不是服务源码里的意图。地址可能配错，服务可能换了版本，或者服务实现返回了错误内容。输入校验保护服务接到的值；客户端内容检查保护后续代码将使用的值，位置和对象都不同。

现有字段检查要求 `id/title/status` 是字符串，`priority/version` 是整数。代码使用 `type(value) is int` 排除 `True` 这类布尔值：Python 中布尔值与整数有继承关系，但这份字段约定不把 `true` 当优先级。这个小检查没有继续证明每一个值的业务含义；例如“编号是否就是刚才创建的那一条”要靠两次结果间的比较，而不是只看它是字符串。

类型正确，是后续可以怎样操作它的依据；是不是本次要找的结果，还需要业务关系来判断。

改一个地方，看看失败从哪里开始

把创建调用中的 `priority=2` 改为 `priority="urgent"`，其他地方不变。这里刻意违反输入约定，用来观察处理位置。

服务会在输入 model 校验处拒绝，返回 `422`，创建函数的业务正文不会运行；客户端收到这个回应后，在状态检查处停止。`created` 没有取得这次的成功返回值，所以后面的取编号、查询和比较也不会发生。

这与“调用在等待时超时”不同。超时只能说明客户端这次没有取得完整可用回应，不能据此断言服务没有创建记录。服务可能已经执行，只是回应没有成功交回。遇到后一种情况，应该先找服务端证据，不要凭猜测重复创建。

再看另一种变化：先把优先级恢复为 `2`，让创建能够成功，再故意把查询行的编号改成 `"T-missing"`。这时第一条创建仍然成立；第二次请求到达查询函数，但找不到这个编号，返回 `404`。程序不是在同一个位置失败，已经发生的创建也不会被后面的查询失败撤销。

这两种变化不需要再写一组相似练习。能沿消息指出“哪个函数有没有运行、哪条后续语句还没机会运行”，就已经抓住了这项任务的主要区别。

两个服务的调用，要观察哪两段？

创建与查询走通以后，可以用本地假摘要服务观察 B04 的两次 HTTP 交换。仍然只用合成文字，不带真实凭据。

先停止前面运行的工单服务：回到它所在窗口按 `Ctrl+C`，等到 PowerShell 提示符回来，再查看 `8000` 的监听是否消失。停止命令和端口实际释放是两份不同的证据。若仍有监听，查清 PID 与所属进程，不根据端口号盲目结束别的程序。

接下来使用两个新开的 PowerShell 窗口运行两个服务，再用第三个窗口发送请求。新窗口仍需进入实验副本。先分别检查 `8001`、`8000` 没有监听，使用的方法与前面的 `8000` 相同。

假供应商窗口运行：

```powershell
& .\.venv\Scripts\python.exe -m uvicorn fake_supplier:app --host 127.0.0.1
  --port 8001
```

工单服务窗口需要明确指定本次测试供应商地址，并确保不携带供应商凭据。在这个专用新窗口里执行：

```powershell
$env:SUPPLIER_BASE_URL = 'http://127.0.0.1:8001'
Remove-Item Env:SUPPLIER_TOKEN -ErrorAction SilentlyContinue
& .\.venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000
```

这里仅改变当前新窗口的进程环境，不修改用户或系统级设置；完成后关闭这个专用窗口即可丢弃本次会话配置。删除的是本次窗口继承的环境变量，不是秘密文件。不要在装着其他正在进行任务的窗口里照搬这段准备。

两个窗口都显示正常启动后，在第三个窗口对应的实验目录新建 `check_summary.py`：

PYTHON

```
import httpx


cases = [
    ("R-OK-42", "Printer offline", 200),
    ("R-FAIL-43", "supplier-500", 502),
    ("R-SLOW-44", "supplier-slow", 504),
    ("R-BAD-45", "x", 422),
]

with httpx.Client(
    base_url="http://127.0.0.1:8000",
    timeout=3.0,
    trust_env=False,
) as client:
    created = client.post(
        "/tickets",
        json={"title": "Printer offline", "priority": 2},
    )
    assert created.status_code == 201
    ticket_id = created.json()["id"]

    for request_id, text, expected_status in cases:
        response = client.post(
            f"/tickets/{ticket_id}/summary",
            headers={"X-Request-ID": request_id},
            json={"text": text},
        )
        assert response.status_code == expected_status
        assert response.headers["X-Request-ID"] == request_id
        print(request_id, response.status_code)
        if response.status_code == 200:
            assert response.json()["summary"] == "SUMMARY Printer offline"
```

`cases` 的每一项放在一对括号中，顺序都是“本次请求 ID、要发的文字、预期状态”。`for` 每次取出一项，把三个位置分别交给三个变量。它只是把同一个调用换四组输入运行，不是框架自动识别失败。

运行：

```powershell
& .\.venv\Scripts\python.exe .\check_summary.py
```

应显示：

```
R-OK-42 200
R-FAIL-43 502
R-SLOW-44 504
R-BAD-45 422
```

这里故意检查具体状态，而不调用 `raise_for_status()`：因为这项实验就是要收到并比较预定的失败回应。B03 使用异常路线，是要阻止业务代码继续使用坏结果；这里使用状态比较，是要检验各条已知分支。动作由当前目的决定。

回看两个服务窗口中的结构化日志。`R-OK-42` 应把工单服务的调用开始、供应商收到请求、工单服务的成功连接起来；`R-FAIL-43` 连接供应商失败与本地 `502`；`R-SLOW-44` 连接读取等待超时与本地 `504`。`R-BAD-45` 对应输入 `"x"`，不满足至少三个字符的要求，应在工单服务的校验处停止，没有这一个 ID 的供应商收到请求事件。

这四个 ID 分别标识四次外部请求。一个 ID 在本次跨服务工作中沿用，不等于把整轮实验的所有请求都写成同一个 ID。并且，日志没打印到屏幕不是绝对的“没有调用”证明；测试里记录供应商调用次数，才能从另一侧核对“422 时零调用”。

`supplier-slow` 让假供应商在回应前等待，而工单服务的 HTTPX Client 在读取等待超过配置边界时超时。本地返回 `504` 后，假供应商可能还会继续完成处理。`504` 是工单服务给外层客户端的回应，不是内层供应商一定返回过一个 `504`。

完成后，在两个服务窗口分别按 `Ctrl+C`；确认提示符恢复，并复查 `8000`、`8001` 的监听已消失，再关闭这些专用窗口。

测试、真实调用和日志，各多看到了什么？

方法	能回答的主要问题
TestClient 和断言	给应用一条指定请求，输入校验、业务逻辑和公开回应是否符合预期
MockTransport	指定 HTTPX 在传输边界得到什么结果，客户端如何分类和处理
本机两个进程的真实调用	进程启动、监听和本机 HTTP 交接能否实际连接起来

方法	能回答的主要问题
request ID 对应的日志	这一次处理在服务内部经过了哪些已记录阶段

它们解决的是不同观察距离的问题。只有想验证网络是否真的接通时，才需要跨实际端口的调用；只是要稳定重现坏 JSON 或特定状态，受控替身更直接。反过来，替身测试通过不能代替服务实际启动。日志帮助解释一次过程，断言帮助比较实际与预期，也不能互相替代。

服务重启后，为什么记录不见了？

目前工单保存在 `app.state.tickets` 指向的内存字典里。`app.state` 是这份应用对象用来挂运行状态的位置；`tickets` 是项目给这份字典起的名字，并不是 FastAPI 自动建立的数据库。

服务进程停止，这份内存状态随之消失。重新运行 `create_app()` 会得到新的空字典和从 42 开始的编号。HTTP 请求与回应可以完全正确，但这不意味着记录已经持久保存。

到这里，程序之间的接口已经能说清、运行和检验。下一册 C 要解决的是另一件事：数据怎样保存、关联、修改，并在一组操作失败时保持一致。B 的内存项目与 C 的数据库练习是独立程序；延续的是能力与问题关系，不是假定它们共享一份数据库或直接接着同一份代码运行。

知识关系总览 | 把消息、函数和证据放回同一张图

学习时按一条请求逐步展开；真正使用时，常常要反过来从问题找到工具。下面不按章节重复步骤，而把容易混淆的关系放到一起。

API、HTTP、JSON，是同一种东西的三个名字吗？

不是。假设一个仓储程序对外提供“查询某件商品库存”的能力：

外部能力与约定：查询库存，要给哪个商品编号，会返回什么
↓ 这项 HTTP API 选择通过 HTTP 交换
请求与回应：GET /stock/P-7 → 200 与结果
↓ 这条回应选择 JSON 表示数据
数据表示：{"product_id":"P-7","available":12}
↓ 调用方解析
程序里的值：Python dict，可读取 available 并继续判断

API 说明怎么使用一项能力；HTTP 规定这次消息交换怎样组织；JSON 表示消息中的数据。API 不一定都通过 HTTP，HTTP body 也不一定都用 JSON。本册把它们组合，是因为这种组合能让不同程序清楚地交换结构化数据。

Python dict 与 JSON 也不是替代工具二选一。dict 在程序内组织值，JSON 在程序边界处表示信息；同一任务可以先后用到两者。`response.json()` 是把表示解析回值，不会自动替程序证明所有字段和业务关系都正确。

选地址、动作还是 body，到底各在决定什么？

设想要读取商品 `P-7` 的库存，再请求为它建立一次补货：

消息中的部分	当前回答的问题	例子
host 与 port	交给哪个正在提供服务的网络入口	本机某个监听地址
path	使用哪个接口或指向哪个资源	`/stock/P-7`
query	本次怎样筛选、展开或呈现结果	`?warehouse=W-2`
method	这次打算读取，还是请求发生变化	`GET` 读取；该接口用 `POST` 创建补货

消息中的部分	当前回答的问题	例子
body	操作需要携带哪些结构化输入	商品编号与补货数量
headers	消息应怎样解释，或附带什么上下文	`Content-Type`、`X-Request-ID`

path、query、body 都可能成为服务函数的参数，但来处与意思不同。参数在函数里都是值，不意味着请求里可以随便交换位置。具体接口说数量来自 JSON body，就不能只往 URL 后面加 `?quantity=10`，期待框架自动把它当成那个字段。

GET 和 POST 也不是按“简单用 GET，复杂用 POST”选择。先看要使用的接口如何约定读取或改变，再看输入放哪里。读取一个很长的结果仍然可以是 GET；一个很短的创建请求仍可能需要 POST。对于没有实现的 method，换个动词不会让服务自动获得相应能力。

看上去都是检查，为什么要放在不同地方？



输入 model 回答“进来的值是否满足约定”，例如优先级能否成为范围内整数。业务判断回答“现在是否允许这件事”，例如目标工单是否存在、修改依据的版本是否过时。公开输出约束回答“这份内部结果能否成为承诺的对外回应”。客户端检查则回答“这一次实际收到的内容能不能交给我的后续代码”。

检查是否可以合并，要看它们检查的对象和时机是否真的相同，不能只看它们都叫校验。

输入不合法时，FastAPI 默认的 `422` 可以发生在 endpoint 正文之前；目标不存在时，业务代码可以主动返回 `404`；输出缺少必需字段时，通常是服务端的实现错误，默认形成 `500`。三者都可能让用户看到失败，但要修复的位置不同。尤其输出失败发生得晚，不能据此推断之前的写入没有发生。

这也是为什么不把整个函数包进一个“出错就返回输入错误”的大 `try`。那样会把服务自身的错误也交给调用者负责，丢掉真正的失败位置。

FastAPI、Uvicorn、HTTPX，谁代替谁？

它们通常是配合，而不是替代。

调用进程

HTTPX 主动发请求

→ HTTP

服务进程

Uvicorn 接收并交给应用
↓
FastAPI 找 route、校验和调用
↓
endpoint 处理具体业务

只调用一个 Python 函数，不会自动经过 HTTP，也不会自动触发整条 FastAPI 请求处理。启动了 Uvicorn，若 route 或输入约定不对，请求仍然可以失败。HTTPX 能正常得到一个 Response，也不代表回应一定是业务成功。

当服务端还需要请求摘要服务时，它会在另一段交换中使用 HTTPX，临时成为调用方。client/server 描述的是这一段交互的角色，不是给某个程序贴上永久身份。

失败时，先看哪里，才不容易改错地方？

现在看见什么	首先要分清的关系
发请求的操作抛出连接或等待异常	这次调用没有交回完整可用 Response；先查地址、监听或对应等待阶段
已收到 `404`、`422`、`502` 等状态	已有回应；看 method、目标、状态与错误内容，而非先怪 JSON parser
`200`，但媒体类型不符合当前约定	状态成功不等于内容约定成立
JSON 能解析，却缺字段或类型不对	解析只建立了值，客户端需要的结构仍未成立
类型都正确，却读回另一条记录	结构正确不等于业务关联正确；检查编号怎样从上次结果进入下次请求
外层收到 `504`	先说明是哪一层返回了它，不能当成内层供应商一定返回过同一个状态

这不是要求遇到每个错误都从第一行重新查。已经得到 Response，就不必否认所有网络交接；输入校验的日志和零调用断言已经成立，就先修输入，不去改供应商。好证据的作用，是缩小下一步需要调查的范围。

`request_id` 能把相关记录找在一起，但它既不是业务工单编号，也不让重复请求自动安全。两次请求可以操作同一条工单，却应该有各自的请求标识。重复请求会不会造成两次业务效果，是后续可靠性设计要解决的另一项问题。

哪些动作值得用熟，哪些细节需要时再查？

经常要自己做出的判断是：这次在读还是在改，值放在哪里，结果在哪一步可继续使用，失败属于哪一段。`GET/POST`、status 与 body 的区别、`json=` 的位置、route 与函数的对应、`raise_for_status()` 和 `response.json()` 各自做什么，会在后面的数据库接口与 AI 调用中不断复用，应逐渐做到能选、能改、能解释，而不是只觉得眼熟。

不必记住所有状态码、所有 timeout 参数和完整框架配置。忘记具体写法时，先确定需要的是哪种作用，再查对应文档。比如知道“要限制等待下一块响应数据”，就能找到 read timeout；知道“要限制对外字段”，就能找到 response model。可查的写法之所以找得回来，依靠的是已经建立的关系，而不是把所有细节留成一团模糊印象。

到这里，系统已经从“一个程序处理自己的值”，扩展成“几个程序按约定交接”。下一步的数据保存、外部依赖恢复和 AI 结果检查，都能挂在这些已有位置上；不需要把每一个新库都当成一门互不相干的知识。

需要继续查阅时

正文主例可以离线理解。以下为官方查用入口，页面会持续更新；运行随书项目时以项目依赖文件与本次实验结果为准，不把网页最新示例当成已安装版本的保证。

- MDN：HTTP 总览、请求方法、状态码：查消息结构、方法与状态的语义。

- Python：JSON：查 Python 值与 JSON 表示的转换。
- FastAPI：First Steps、Request Body、Response Model、Testing：查 route、输入、输出与接口测试。
- Pydantic：Models、Settings：查模型值、类型处理与运行配置。
- HTTPX：QuickStart、Clients、Timeouts、Transports：查主动调用、连接复用、等待边界和受控传输。
- Uvicorn：Settings：查服务启动与监听参数。
- FastAPI：Header 参数、错误处理、回应 Header，以及 Python：Logging HOWTO：查关联标识与事件记录所用的具体入口。

如果浏览器和脚本的表现不一致，而且已经确认请求本身相同，再进入浏览器缓存、跨来源限制等按需材料；如果服务出现重复前置检查或多个独立接口组，才需要继续查共享依赖和 router。它们是遇到具体压力后的扩展，不影响这里的基本交接关系成立。

本册总览（需要时再回看）

这里保留整册地图、章节关系和查用入口；第一次阅读不需要先读完。

本册知识结构总览

下面是同一次程序交接的几个观察角度，不是必须依次背下来的五个类别。

两个程序怎样一起完成一件事？	
交换约定：怎样表达“请做什么”和“做得怎样”	URL、method：目标在哪里，要对它做什么 headers、body：说明怎样解释消息，携带实际数据 status、回应内容：处理结果是什么，返回了什么
数据表示：Python 的值怎样变成双方都能读的 JSON	
接收与处理：请求怎样进入函数，谁检查输入和业务条件	
调用与使用：怎样发送请求，怎样拒绝不可用的回应	
观察与验证：怎样从报文、日志和测试找回实际发生的事	

这些关系会组合在一起。例如，创建工单时，URL 和 method 指出创建入口，JSON body 带来标题与优先级；服务端检查这些值后建立记录，回应把新编号交回。调用方再把这个编号放进下一次查询的地址。**上一次的结果，能成为下一次请求的输入；两边的程序并不共享同一个 Python 变量。**

FastAPI、HTTPX、Uvicorn 和浏览器也会陆续出现。先知道它们不是四套互相竞争的 HTTP：FastAPI 帮助接收方把请求交给 Python 函数，HTTPX 帮助 Python 主动发起调用，Uvicorn 让服务实际运行并监听，浏览器的 Network 面板用来观察浏览器发出的请求。具体怎么做，会在需要它的时候讲。

从哪里读到哪里

章节	要回答的问题
B01	另一个程序怎样收到工单，并把创建结果交回？
B02	收到请求后，为什么会运行这个 Python 函数？
B03	拿到回应以后，怎样判断结果可以继续用？
B04	一次工作经过两个服务，怎样找到最早失败的位置？
B05	把服务与客户端实际放在一起，能否创建后再读回同一条记录？

先读例子中的输入和结果就能理解主线；B05 集中给出电脑上的准备与运行方法。不需要一边首次认识 HTTP，一边提前配置所有工具。各章都有足以追踪当前问题的上下文，B02 的小程序也会从头给出，不要求记得某份旧练习。

这里不要求默写整份服务代码。值得逐渐用熟的是少量经常发生的动作：读完整地址和 method，查看 status 与 body，用 `json=` 提交结构化数据，用 route 和输入 model 接入函数，把异常检查放在对应操作周围。更重要的是能解释这些动作为什么在那里。完整配置、少用的参数和错误码细节，在实际需要时查用即可。

本册后部的“知识关系总览”会按关系再整理一次：哪些工具各管一段，哪些检查看似重复却不能互相替代，以及什么时候选择哪一个。第一次阅读先认识要探索的范围，需要时再回看收束。